# GRASP: Gradient-Guided Sampling for Contact-Rich Physics-Based Motion Retargeting

**Yang Liu**[1*], **Tong Wang**[1*], **Yiming Bao**[1], **Heng Yang**[1]

[1]School of Engineering and Applied Sciences, Harvard University
[*]Equal contribution

We study physics-based retargeting of human demonstrations for robot learning: finding robot controls that reproduce reference motions under contact dynamics. We propose GRASP (Gradient Recovery from Archived Samples for Proposals), which combines sampling-based exploration with gradient-guided refinement without requiring a differentiable simulator. GRASP-TO extends tempered sequential Monte Carlo by fitting local quadratic cost models to archived rollouts and using their gradients to guide Markov chain Monte Carlo rejuvenation. Metropolis–Hastings correction preserves the target distribution for each fixed proposal kernel, and a population analysis connects the recovered gradients to Gaussian smoothing. GRASP-MPC scales this optimizer to long demonstrations through warm-started receding-horizon planning. Experiments on dexterous hands and humanoids show accurate retargeting with a shared tracking objective and no task-specific contact rewards, including 94.2% success on 1352 bimanual OakInk2 demonstrations. The refined references improve PPO tracking in pen spinning and humanoid–object interaction. We also demonstrate dexterous manipulation and whole-body humanoid motion on real hardware.

**Keywords:** **Physics-based motion retargeting, trajectory optimization, sequential Monte Carlo, model predictive control, dexterous manipulation, humanoid**

**Website:** https://computationalrobotics.seas.harvard.edu/GRASP

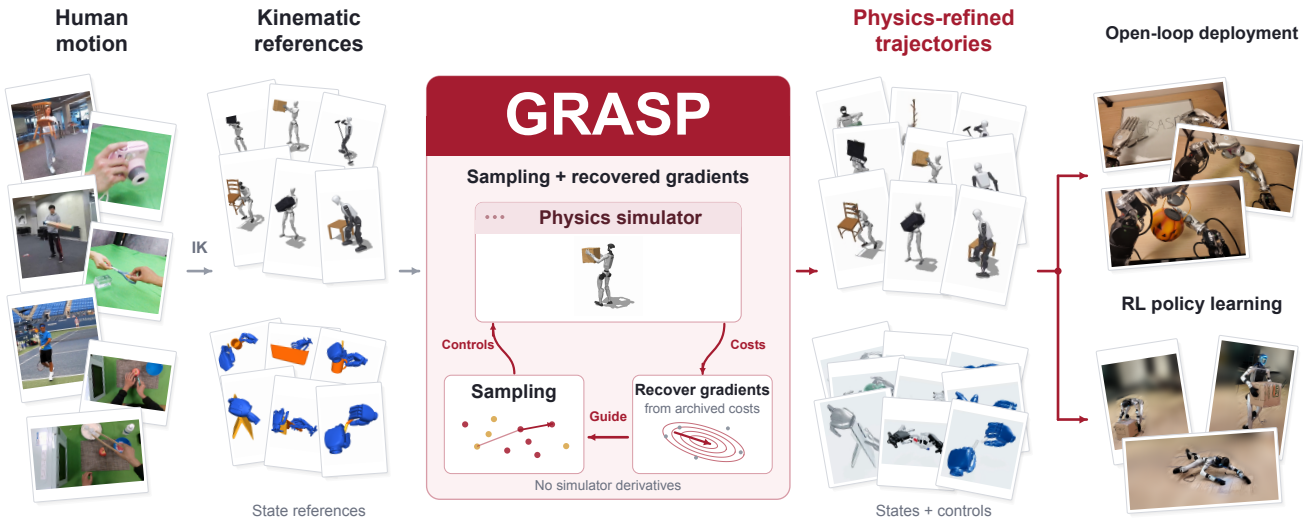


**Figure 1 GRASP in the human-to-robot pipeline.** GRASP combines sampling and recovered gradients through simulator feedback, refining kinematic references for open-loop deployment and RL tracking.

# 1 Introduction

Human demonstrations provide a rich source of reference motions for robot learning, capturing coordinated movements and interactions that would be difficult to specify by hand. Retargeting these demonstrations to robots with different body geometries and actuation capabilities provides references for expressive humanoid control [5, 14, 15] and dexterous manipulation [31]. The quality of the resulting references matters: motions that a robot can physically realize provide a stronger basis for learning reliable tracking policies.

Retargeting methods broadly fall into two categories. *Kinematic retargeting* maps human motion onto robot configurations, accounting for differences in geometry and joint limits and, in some methods, preserving spatial relationships with objects and terrain [1, 43]. These geometric constraints alone do not ensure that the robot can generate the forces needed to execute the motion. A hand may follow an object without establishing a grasp, or a humanoid may attempt a movement that its ground contacts cannot support. *Physics-based retargeting*, the focus of this paper, addresses this gap by optimizing robot actions in simulation [8, 26, 27, 44]. It seeks to reproduce demonstrated movements and interactions while satisfying the simulated dynamics, making it especially important for contact-rich manipulation and whole-body motion.

Physics-based retargeting optimizes robot controls to track robot-state references derived from human motion and, when present, demonstrated object motion, subject to the dynamics of a chosen physics simulator (cf. Section 3). Two common approaches to this trajectory optimization problem are sampling-based search and gradient-based nonlinear programming. Sampling methods such as MPPI and cross-entropy-method variants evaluate populations of control sequences through simulator rollouts [29, 34, 41]; they support broad exploration and have enabled large-scale physics-based retargeting [26, 27]. Gradient-based methods exploit local sensitivities and can offer faster local convergence. For example, Esteban et al. [9] formulate dynamic motion retargeting as a multiple-shooting nonlinear program using simulator transition derivatives. These approaches offer complementary advantages, but combining them requires useful gradient information.

Contact dynamics make this requirement difficult to meet. A small control change can create or remove a contact and substantially alter the subsequent motion. Many simulator interfaces expose only forward rollouts; even when derivatives are available through regularization, finite differences, or automatic differentiation, they can vary sharply or fail to reveal improvements across contact modes [35, 37]. This motivates our central question:

> *Can we achieve high-performance physics-based retargeting by combining sampling-based exploration and gradient-guided refinement without assuming differentiable simulator dynamics?*

**Contributions.** Tempered sequential Monte Carlo (TSMC) provides a principled framework for this combination: importance reweighting and resampling progressively favor low-cost trajectories, interleaved with gradient-guided Markov chain Monte Carlo (MCMC) rejuvenation [42]. Its existing trajectory optimization formulation, however, obtains those gradients by differentiating through the simulator and therefore does not directly provide gradient-guided rejuvenation for nondifferentiable dynamics. We introduce GRASP (Gradient Recovery from Archived Samples for Proposals), which supplies this guidance by reusing the rollout evaluations already produced during search. Our contributions are fourfold:

- **GRASP-TO: trajectory optimization from recycled rollouts.** We archive evaluated trajectory–cost pairs, including rejected proposals, and fit local quadratic cost models in a reduced temporal basis. Their surrogate gradients guide MCMC rejuvenation without additional simulator queries for fitting. We establish that, for deterministic rollout costs and fixed surrogate and proposal parameters, the Metropolis–Hastings correction preserves the original tempered target distribution despite surrogate fitting errors. We also show that, under a Gaussian fitting measure, the unregularized population quadratic recovers the gradient and Hessian of a Gaussian-smoothed cost at the fitting center, connecting rollout regression to randomized smoothing [25, 35].
- **GRASP-MPC: retargeting long demonstrations.** We extend GRASP-TO to a receding-horizon formulation that replans from the simulated states actually reached. Shifted-plan warm starts, candidate reuse, and alternating full-search and refinement windows reduce repeated work. Joint optimization of the initial hand configuration and first motion window couples the starting grasp to the ensuing interaction. This applies the hybrid optimizer to long retargeting problems using a standard state-tracking objective without task-specific contact rewards.

- **Extensive simulation and policy-learning evaluation.** We evaluate GRASP-TO on smooth and contact-switching optimization benchmarks and GRASP-MPC on dexterous-hand and humanoid retargeting. On 1352 OakInk2 demonstrations with Sharpa hands, we achieve 94.2% success, exceeding the published reference success rates; on 5,400 HOT3D trajectories, we achieve 81.54% ADD-AUC. The same hand-retargeting configuration supports Wuji and XHand with success rates above 92%. On LAFAN1, AMASS, and OMOMO, we obtain lower joint and pelvis-translation errors than the published reference results. Controlled PPO comparisons show that the refined references improve downstream learning and tracking for both pen spinning and humanoid–object interaction.
- **Real-world dexterous hands and humanoid execution.** We demonstrate open-loop playback of GRASP-generated trajectories on physical dexterous hands and closed-loop execution of learned policies trained to track refined references on humanoids. Demonstrations include lifting and pouring with hands, and crawling, box carrying, and box pushing with humanoids, showing how the refined motions support both trajectory playback and policy learning for real-world interaction.

**Outline.** Sections 2 and 3 review related work and formulate physics-based retargeting. Sections 4 and 5 develop GRASP-TO and GRASP-MPC. Sections 6 and 7 evaluate dexterous-hand and humanoid retargeting, including policy learning and real-robot execution. Section 8 concludes. Appendices A–D provide the prior, proofs, and evaluation details; Appendix E reports the GRASP-TO benchmarks.

## 2 Related Work

**Motion retargeting.** Kinematic retargeting adapts human motion to robot geometry, providing references for humanoid control in H2O [14] and GMR [1]. OmniRetarget further preserves robot–object and robot–terrain relationships during whole-body retargeting [43]. Physics-based methods refine these references by optimizing robot actions under simulated dynamics. PhysicsGen combines kinematic retargeting and trajectory optimization across embodiments [44], while DynaRetarget uses sampling-based trajectory optimization for dynamically feasible motion references [8]. Several approaches assist contact discovery with specialized guidance: SPIDER uses virtual contact constraints to guide retargeting for hands and humanoids [27], and Do-as-I-Do augments sampling-based dexterous retargeting with rewards for pick-and-place transitions [26]. Shooting for Contact [9] instead uses a multiple-shooting nonlinear program with simulator derivatives to retarget humanoid motion without prescribing contact schedules. GRASP uses a common robot-and-object state-tracking objective without virtual contact guidance or task-specific contact rewards. The optimizer discovers the contacts needed to reproduce the demonstrated interaction through forward simulation, without requiring dynamics derivatives.

**Optimization through contact.** Three broad approaches address the nonconvexity and nonsmoothness of contact-rich trajectory optimization. *Local gradient-based methods* include contact-invariant optimization [24] and the direct method of Posa et al. [30], which jointly optimizes motion and contact forces through a complementarity-constrained nonlinear program solved by sequential quadratic programming. More recent methods include CRISP's sequential convex optimization [22], IMPACT's augmented-Lagrangian formulation [21], and the Contact Trust Region [36]. These methods can provide efficient local refinement, but require derivative information and can remain sensitive to initialization and local minima. *Sampling-based methods*, such as MPPI [41] and iCEM [29], explore candidate trajectories without differentiating the dynamics. However, local convergence can be slow and require many rollouts in high-dimensional control spaces. *Global optimization through convex relaxations* provides lower bounds on the optimal cost and can produce feasible plans with certified suboptimality gaps. Graesdal et al. [10] combine semidefinite relaxations with graphs of convex sets for contact-mode selection in quasi-static planar pushing. Kang et al. [18] exploit polynomial and robotics-specific sparsity to construct higher-order semidefinite relaxations for contact-rich planning. For polynomial trajectory optimization, STROM combines sparse semidefinite relaxations with a GPU-accelerated solver [11, 19]. Despite substantial gains from sparsity and parallelization, large relaxations remain computationally expensive as the problem dimension and number of contact modes grow. GRASP combines sampling-based exploration with surrogate-gradient refinement. Its optimizer requires only batched rollout costs, allowing it to operate with any simulator that provides this interface and to address long, high-dimensional demonstrations through receding-horizon optimization.

**Sampling-based optimization and gradient recovery.** Stochastic optimizers use rollout populations in different ways: STOMP updates trajectories using noisy rollout costs [17], and CBO studies global trajectory optimization through interacting particles [38]. Sequential Monte Carlo evolves a population through intermediate target distributions [7];

TSMC adapts this framework to trajectory and policy optimization by combining tempering with rejuvenation guided by simulator gradients [42]. Other hybrids combine CEM with gradient descent in MPC [4] or guide MPPI with stochastic surrogate gradients, as in SVG-MPPI [16]. When analytic derivatives are unavailable, local surrogate models provide search directions in derivative-free optimization [6], while randomized smoothing connects function evaluations to gradients of smoothed objectives [25, 35, 37]. NFG estimates natural functional gradients of a Gaussian-smoothed trajectory objective from cost evaluations [28], and WASP reuses information across successive derivative estimates [32]. GRASP integrates rollout reuse into tempered population search: local quadratic fits to archived costs, including rejected proposals, guide MCMC rejuvenation, while Metropolis–Hastings acceptance uses the true simulator costs. This couples exploration and local refinement without additional simulator queries for fitting or access to dynamics derivatives.

# 3 Problem Formulation: Physics-Based Motion Retargeting as Dynamics-Constrained Trajectory Tracking

Physics-based motion retargeting seeks robot controls that reproduce a human demonstration under simulated dynamics. A kinematically retargeted demonstration provides reference states $\bar{x}_{0:T} = (\bar{x}_0, \dots, \bar{x}_T)$ over $T$ simulation steps. At step $t$, the state is $x_t = (q_t, v_t) \in \mathcal{X}$, where $q_t \in \mathcal{Q}$ is the combined robot and object configuration, $v_t \in \mathbb{R}^{n_v}$ contains their generalized velocities, and $\mathcal{X} = \mathcal{Q} \times \mathbb{R}^{n_v}$. The configuration space $\mathcal{Q}$ includes robot joint positions and robot and object poses. The robot command $u_t \in \mathbb{R}^m$ has $m$ control channels; in our retargeting experiments, these commands are position targets. The simulator transition $f : \mathcal{X} \times \mathbb{R}^m \to \mathcal{X}$ includes the actuation and contact dynamics.

Given an initial state $x_{\text{init}}$, we optimize the command sequence $u_{0:T-1} = (u_0, \dots, u_{T-1})$ to track the demonstration:

$$\begin{aligned} \min_{u_{0:T-1}} \quad & J(u_{0:T-1}) = \sum_{t=1}^{T} \ell(x_t; \bar{x}_t) + w_{\text{term}} \ell(x_T; \bar{x}_T), \\ \text{s.t.} \quad & x_{t+1} = f(x_t, u_t), \quad t = 0, \dots, T-1, \\ & x_0 = x_{\text{init}}. \end{aligned} \tag{1}$$

Here $\ell : \mathcal{X} \times \mathcal{X} \to \mathbb{R}_{\geq 0}$ is the stage tracking loss, and $w_{\text{term}} \geq 0$ gives additional weight to tracking at the horizon endpoint. We use the same loss for the stage and terminal contributions:

$$\ell(x_t; \bar{x}_t) = \|W_q(q_t \ominus \bar{q}_t)\|_2 + w_v \|v_t - \bar{v}_t\|_2, \tag{2}$$

where $\bar{x}_t = (\bar{q}_t, \bar{v}_t)$. The operator $\ominus$ forms a configuration-error vector in $\mathbb{R}^{d_q}$, using relative rotations for orientation components. The diagonal matrix $W_q \in \mathbb{R}^{d_q \times d_q}$ assigns nonnegative weights to object pose, robot base pose, and joint errors, while the scalar $w_v \geq 0$ weights velocity errors. The kinematic reference need not satisfy the simulator dynamics. Objects are unactuated and move under simulated contact and gravity, so reproducing their demonstrated motion requires the optimizer to discover the necessary contacts. Changes in the controls can alter these contacts, making the rollout cost nonsmooth even with a simple tracking loss.

To reduce the number of decision variables, we parameterize the commands by $K$ control-offset knots $z_k \in \mathbb{R}^m$ and stack them into $z = (z_0^\top, \dots, z_{K-1}^\top)^\top \in \mathbb{R}^{Km}$:

$$u_t(z) = \bar{u}_t + \sum_{k=0}^{K-1} \psi_k(t)\, z_k, \qquad \psi_k(t) = \max\bigl(0,\, 1 - |\min(t/\Delta, K-1) - k|\bigr). \tag{3}$$

where $\bar{u}_t \in \mathbb{R}^m$ contains the reference position targets, we use $K$ knots at $t_k = k\Delta$, $k = 0, \dots, K-1$, with $\Delta = T/K$. We linearly interpolate between knots and hold $z_{K-1}$ over the final interval $[T - \Delta, T)$. Figure 2 illustrates this parameterization for one control channel. We write $J(z) = J(u_{0:T-1}(z))$ for the resulting objective on $\mathbb{R}^{Km}$. Our optimizer accesses this objective only through batched simulator rollouts; it requires no derivatives of $f$.

We focus on two settings, distinguished by whether the controls are replanned as the motion unfolds.

**Open-loop trajectory optimization (TO).** In the open-loop case, we solve Eq. (1) once from $x_{\text{init}}$ and execute the resulting command sequence without replanning. Each candidate is evaluated over the full horizon, so both the rollout length and the number of knots grow with the demonstration duration.

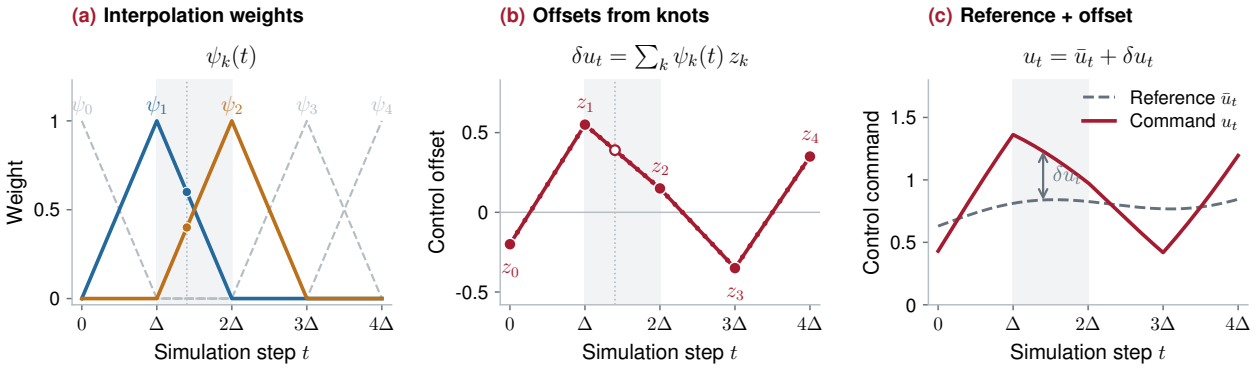


**Figure 2** Control-knot parameterization for one control channel, with $K = 5$ and illustrative values. (a) Triangular weights interpolate between neighboring knots; the shaded interval highlights two overlapping weights. (b) The optimized offsets $z_k$ (large dots) define an offset at every simulation step by linear interpolation. (c) Adding this offset to the reference command produces $u_t$. The interpolation applies componentwise to all $m$ control channels. The plot ends at the last knot; its offset is held over the final interval.

**Closed-loop model predictive control (MPC).** In the closed-loop case, we solve the same tracking problem over a shorter window initialized at the current simulated state, with the terminal contribution applied at the window endpoint. We execute only a prefix of the selected command sequence and then replan from the state reached. This receding-horizon procedure closes the feedback loop around the simulated motion during retargeting, limits the size of each search, and adapts subsequent commands to the contacts actually realized.

Section 4 develops GRASP-TO for the open-loop problem. Section 5 then uses this solver within GRASP-MPC to retarget long demonstrations through closed-loop replanning.

# 4 GRASP for Trajectory Optimization

We first develop the open-loop solver, which searches over the knot offsets $z \in \mathbb{R}^{Km}$. We use a *single-shooting* formulation: each candidate defines the command sequence $u(z) := u_{0:T-1}(z)$ through Eq. (3), and the simulator unrolls $x_0(z) = x_{\text{init}}$ and $x_{t+1}(z) = f(x_t(z), u_t(z))$ over the full horizon. Substituting these states into Eq. (1) makes the entire cost a function of $z$ alone; we use the shorthand $J(z) := J(u(z))$. Thus, the states are determined by the rollout, and the dynamics are satisfied by construction. The solver accesses $J$ through batched simulations.

## 4.1 Preliminaries: Tempered Sequential Monte Carlo

We review tempered sequential Monte Carlo (TSMC) [7, 42], which turns trajectory optimization into sampling from a sequence of distributions that increasingly favor low-cost controls.

**From optimization to inference.** We place a Gaussian autoregressive (AR(1)) prior $p_0(z)$ on the knot offsets, independently per control channel, with coefficient $\rho \in [0, 1)$ and positive per-channel, per-knot exploration scales (Appendix A). This prior has full support on $\mathbb{R}^{Km}$ and favors temporally smooth perturbations of the reference controls. Following Yang [42], consider the distributional objective

$$\min_p \ \mathbb{E}_{z\sim p}[J(z)] + \beta^{-1} D_{\text{KL}}(p\|p_0), \tag{4}$$

where $p$ ranges over probability densities, $D_{\text{KL}}$ is the Kullback–Leibler divergence, and $\beta > 0$ is the inverse temperature. Its minimizer is the Boltzmann-tilted density $\pi_1$ in the path

$$\pi_\lambda(z) = \frac{1}{Z_\lambda} p_0(z) \exp[-\lambda\beta J(z)], \qquad 0 \le \lambda \le 1. \tag{5}$$

Here $Z_\lambda$ is the normalizing constant: $\pi_0 = p_0$, and $\pi_1$ is the desired inference target. As $\beta \to \infty$, $\pi_1$ concentrates on arbitrarily near-optimal costs, provided every sublevel set $\{z : J(z) \le J_* + \delta\}$ has positive prior probability for $\delta > 0$, where $J_* := \inf_z J(z)$. Tempering advances $\lambda$ from zero to one at a fixed $\beta$ to approach this potentially sharp, multimodal target gradually.

**Importance weighting and resampling.** TSMC maintains $N$ candidate knot vectors, called *particles*, initialized as independent draws $z^{(i)} \sim p_0$, $i = 1, \dots, N$. The superscript indexes particles, whereas the subscript in $z_k$ indexes knots within a particle. Given an equally weighted population approximating $\pi_\lambda$, a proposed increase to $\lambda' \in (\lambda, 1]$ yields incremental importance weights

$$\widetilde{w}_i = \exp[-(\lambda' - \lambda)\beta J(z^{(i)})], \qquad w_i = \frac{\widetilde{w}_i}{\sum_{j=1}^{N} \widetilde{w}_j}. \tag{6}$$

These weights are proportional to the density ratio $\pi_{\lambda'}(z^{(i)})/\pi_\lambda(z^{(i)})$ and favor lower-cost particles. Their concentration is measured by the effective sample size

$$\mathrm{ESS}(\lambda') = \frac{1}{\sum_{i=1}^{N} w_i^2} \in [1, N]. \tag{7}$$

An ESS of $N$ means uniform weights; an ESS near one means that a single particle dominates. We choose the largest tempering increment that keeps ESS at least $\gamma N$, for a prescribed fraction $\gamma \in (0, 1)$. Resampling then draws $N$ particles with replacement according to $w_i$ and resets their weights to $1/N$, allocating more computation to promising trajectories.

**MCMC rejuvenation.** Resampling can create repeated particles. Markov chain Monte Carlo (MCMC) moves diversify these copies and explore nearby controls while leaving the new target $\pi_{\lambda'}$ invariant. Repeating weighting, resampling, and rejuvenation advances the population to $\lambda = 1$. The differentiable-dynamics method of Yang [42] uses Hamiltonian Monte Carlo (HMC) with gradients through simulator rollouts. Here we use the simpler Metropolis-adjusted Langevin algorithm (MALA) [33] with a Metropolis–Hastings correction [13]. At a fixed tempering level $\lambda$, its gradient-based proposal is

$$\begin{aligned} z' &= z + \tfrac{\epsilon^2}{2} M g(z) + \epsilon M^{1/2} \xi, \qquad \xi \sim \mathcal{N}(0, I_{Km}), \\ g(z) &= \nabla \log \pi_\lambda(z) = \nabla \log p_0(z) - \lambda\beta \nabla J(z), \end{aligned} \tag{8}$$

where $\epsilon > 0$ is the step size and $M \in \mathbb{R}^{Km \times Km}$ is a positive-definite, prior-based preconditioner. Writing $Q_\lambda(z' \mid z)$ for this Gaussian proposal density, we accept the proposed particle with probability

$$\alpha(z, z') = \min\left\{1, \frac{p_0(z') e^{-\lambda\beta J(z')} Q_\lambda(z \mid z')}{p_0(z) e^{-\lambda\beta J(z)} Q_\lambda(z' \mid z)}\right\}, \tag{9}$$

and otherwise retain $z$.

**Asymptotic guarantees.** For a fixed tempering schedule, target-invariant Markov kernels, and standard resampling and integrability conditions, TSMC is consistent: as $N \to \infty$, particle averages of bounded test functions converge in probability to their expectations under the corresponding $\pi_\lambda$ [7, 42]. ESS-adaptive schedules also admit consistency results under additional regularity conditions [3]. This large-population limit controls sampling error at fixed $\beta$; the separate limit $\beta \to \infty$ concentrates the target near optimal costs.

The TSMC weighting and resampling steps require only cost values, and the acceptance test combines true costs with the proposal densities. The simulator-gradient requirement enters through the proposal drift $\nabla J(z)$, computed by differentiating the single-shooting rollout. Many contact simulators do not provide these derivatives, and even when available they can be unreliable near contact transitions [35]. Gradient-free random-walk kernels remain valid but can mix slowly in the high-dimensional knot space. Section 4.2 develops GRASP to recover useful proposal directions from rollout values without assuming differentiable dynamics.

## 4.2 Recycling Rollouts into Gradients

GRASP replaces derivatives through the simulator with derivatives of a local model fitted to previously evaluated controls. The same rollouts therefore serve two purposes: they identify promising trajectories and reveal how the cost varies around the current population. Figure 3 illustrates this reuse, and Algorithm 1 summarizes the resulting search.

**An archive and a local coordinate system.** Let $d := Km$ be the dimension of a control vector. An archive $\mathcal{D} = \{(z^{(j)}, J(z^{(j)}))\}_{j=1}^{n_\mathcal{D}}$ stores $n_\mathcal{D}$ evaluated controls and their costs, including proposals rejected by the Markov kernel. Thus, $N$ denotes the current population size, while $n_\mathcal{D}$ counts archived evaluations. The population determines

a center $\mu \in \mathbb{R}^d$ and a diagonal matrix $D \in \mathbb{R}^{d\times d}$ with positive entries that scale each control coordinate. We retain $1 \le r \le K$ low-frequency discrete cosine transform components per control channel, collected in an orthonormal basis $B \in \mathbb{R}^{d\times d_r}$, where $d_r := rm$ and $B^\top B = I_{d_r}$. The reduced coordinates and quadratic model are

$$y(z) = B^\top D^{-1}(z-\mu) \in \mathbb{R}^{d_r}, \qquad \widehat{F}(y) = a + b^\top y + \tfrac{1}{2} y^\top A y, \tag{10}$$

with intercept $a \in \mathbb{R}$, slope $b \in \mathbb{R}^{d_r}$, and symmetric curvature matrix $A \in \mathbb{R}^{d_r\times d_r}$. The low-frequency basis keeps the regression manageable while retaining coordinated changes over time.

**A weighted quadratic fit.** Following the local-model perspective of derivative-free optimization [6], we estimate these coefficients by weighted ridge regression:

$$\min_{a,b,A=A^\top} \sum_{j=1}^{n_{\mathcal{D}}} \omega_j \left[J(z^{(j)}) - \widehat{F}(y^{(j)})\right]^2 + \eta \mathcal{R}(b,A), \qquad y^{(j)} := y(z^{(j)}). \tag{11}$$

The proximity weights $\omega_j \ge 0$ emphasize archived controls near the current population in standardized coordinates. They are regression weights, distinct from the TSMC importance weights $w_i$ in Eq. (6). The parameter $\eta > 0$ controls a quadratic penalty $\mathcal{R}$ on the slope and curvature coefficients. For finite cost values and positive total weight, a positive-definite ridge penalty makes the fit unique even when the samples do not determine every coefficient. Neither the regression nor its well-posedness requires $J$ to be continuous or differentiable.

We distinguish the reduced model $\widehat{F}$ from its extension to control space, $\widehat{J}(z) := \widehat{F}(y(z))$. Its gradient follows by the chain rule:

$$\nabla_z \widehat{J}(z) = D^{-1} B\big(b + A y(z)\big). \tag{12}$$

Fitting uses only archived rollout values and requires no additional simulator calls. The center, scales, basis, and fitted coefficients are held fixed throughout the associated block of Markov moves.

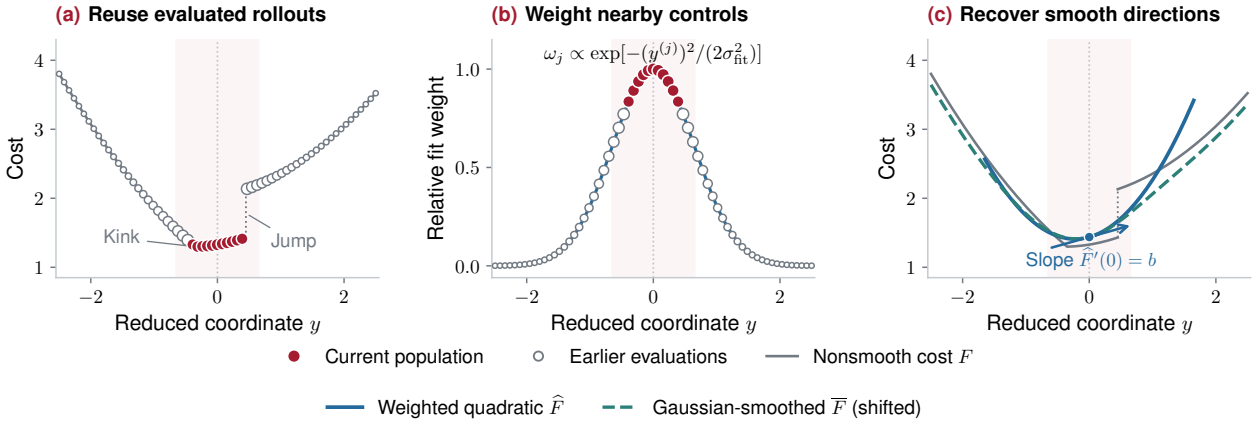


**Figure 3** Recovering a smooth proposal direction from nonsmooth rollout costs, illustrated with one reduced coordinate. (a) Current particles (filled crimson) and earlier evaluations (open gray) both belong to the archive; the cost has a kink and a jump. (b) Illustrative Gaussian proximity weights with width $\sigma_{\text{fit}}$ favor samples near the current population center $y = 0$; shading marks $|y| \le \sigma_{\text{fit}}$, and marker area indicates regression weight. (c) The weighted quadratic fit is differentiable despite these cost transitions. The Gaussian-smoothed cost is shifted vertically to match the fitted value at zero, allowing their local slopes and curvatures to be compared. Proposition 1 gives the exact derivative identities for the ideal Gaussian population fit. All values are illustrative.

**Guidance with a Metropolis correction.** We replace the cost gradient in Eq. (8) with the recovered direction,

$$\widehat{g}(z) = \nabla \log p_0(z) - \lambda\beta\, \nabla_z \widehat{J}(z). \tag{13}$$

The resulting Gaussian proposal density $Q_\lambda(z' \mid z)$ must use this same surrogate drift in both directions of the acceptance ratio in Eq. (9). The simulator evaluates the true cost $J(z')$, which enters that ratio and is appended to the archive whether the proposal is accepted or rejected.

For any fixed surrogate, preconditioner, and step size, the corrected kernel is reversible with respect to $\pi_\lambda$, regardless of the surrogate's fitting error (Appendix B). Thus, model error can affect the efficiency of proposals without changing

**Algorithm 1** GRASP: trajectory search from rollout feedback

**Require:** Cost $J$, prior $p_0$, inverse temperature $\beta$, particle count $N$, ESS fraction $\gamma$
Draw $z^{(i)} \sim p_0$; evaluate $J_i = J(z^{(i)})$, $i = 1, \dots, N$
Initialize $\mathcal{D} = \{(z^{(i)}, J_i)\}_{i=1}^{N}$; set $\lambda = 0$
**while** $\lambda < 1$ and stage budget remains **do**
    Choose the largest $\lambda' \in (\lambda, 1]$ with $\mathrm{ESS}(\lambda') \geq \gamma N$
    Resample particles and their costs using Eq. (6)
    Set $\lambda \leftarrow \lambda'$
    Fit $\widehat{F}$ from $\mathcal{D}$; freeze $\mu, D, B, a, b, A, M$, and $\epsilon$
    **for** each Markov move **do**
        Propose $z'^{(i)}$ using Eqs. (8) and (13)
        Evaluate $J(z'^{(i)})$ in parallel physics rollouts
        Add every $(z'^{(i)}, J(z'^{(i)}))$ to $\mathcal{D}$
        Accept or reject with Eq. (9); update $z^{(i)}$ and $J_i$
    **end for**
**end while**
**return** Candidate trajectories and their rollout costs

the invariant distribution of that fixed kernel. Between move blocks, new archived rollouts can update the model and improve subsequent guidance.

### 4.3 Gradient Recovery as Randomized Smoothing

A quadratic fit always provides a smooth proposal field, but which features of the nonsmooth cost does that field capture? We connect the population regression to randomized smoothing [25, 35], separating this exact characterization from the finite, regularized fit used in practice.

**The cost seen in reduced coordinates.** Fix $\mu, D, B$, and let $\nu_z$ denote a population fitting measure on control vectors: it represents both the locations of archived samples and their proximity weights. For $Z \sim \nu_z$, write $Y := y(Z)$ and let $\nu$ be the distribution of $Y$. Assuming $\mathbb{E}_{\nu_z}[J(Z)^2] < \infty$, define the reduced response

$$F(y) := \mathbb{E}_{\nu_z}[J(Z) \mid Y = y]. \tag{14}$$

This conditional mean accounts for variations in the coordinates that $B$ discards. If all sampled controls lie on the slice $Z = \mu + DBY$, it reduces to $F(y) = J(\mu + DBy)$. Least-squares fitting to the full rollout costs is equivalent to fitting this response: the residual variance conditional on $Y$ contributes a constant independent of the quadratic coefficients. The unregularized population problem is therefore

$$(a_*, b_*, A_*) = \arg\min_{a,b,A=A^\top} \mathbb{E}_{Y\sim\nu}\left[\left(F(Y) - a - b^\top Y - \tfrac{1}{2}Y^\top AY\right)^2\right]. \tag{15}$$

A Gaussian reduced fitting measure yields an exact interpretation of its slope and curvature coefficients.

**Proposition 1** (Derivatives of a Gaussian-smoothed cost)**.** *Let $\nu = \mathcal{N}(0, \Sigma)$ with $\Sigma \in \mathbb{R}^{d_r \times d_r}$ positive definite, and let $F : \mathbb{R}^{d_r} \to \mathbb{R}$ be measurable with at most polynomial growth. Define the Gaussian convolution*

$$\overline{F}(y) := \mathbb{E}_{Y\sim\nu}[F(y + Y)]. \tag{16}$$

*The unique minimizer of Eq. (15) satisfies*

$$b_* = \nabla\overline{F}(0), \qquad A_* = \nabla^2\overline{F}(0). \tag{17}$$

The proof uses Gaussian polynomial orthogonality and differentiation of the convolution kernel (Appendix B). It does not require derivatives of $F$.

The fitted gradient $b_* + A_* y$ is consequently the first-order Taylor approximation of $\nabla\overline{F}(y)$ at the population center. Gaussian convolution smooths both kinks and jumps, so neighboring cost values can reveal a useful local direction

even when the original response has no derivative; Figure 3(c) illustrates this effect. The covariance $\Sigma$ sets the smoothing scale in reduced coordinates; $D$ and $B$ determine how these coordinates relate to control perturbations.

The proposition is a population identity under a Gaussian fitting measure. A finite archive, ridge regularization, and a non-Gaussian fitting measure can all change the recovered coefficients. It therefore explains how value-based regression can supply smooth guidance without identifying every empirical fit with an exact Gaussian convolution. As the search concentrates, fitting over smaller neighborhoods can resolve finer cost structure; the Metropolis correction continues to use the original rollout cost at every stage.

**Optimizer validation.** We validate GRASP-TO on acrobot swing-up and a contact-switching pusher-slider task. To keep the main experiments focused on motion retargeting, we defer these optimizer benchmarks and baseline comparisons to Appendix E. We next extend the solver to long demonstrations through receding-horizon optimization.

# 5 GRASP for Model Predictive Control

GRASP-TO searches over an entire control sequence from a given initial state. For long demonstrations, both the number of decision variables and the cost of each rollout grow with the clip duration. An early tracking error can also change subsequent contacts and invalidate the remainder of an open-loop plan. GRASP-MPC addresses these challenges through the receding-horizon formulation of Section 3: it optimizes a bounded window, executes a short prefix, and replans from the simulated state actually reached. We combine this feedback with plan and particle reuse to avoid repeating a full search at every window. Figure 4 illustrates the overlapping windows, state feedback, and reuse of unexecuted controls.

## 5.1 Windowed Tracking and Initial Configuration

Let $t < T$ be the current simulation step and $H_t \in \{1, \dots, T-t\}$ the planning horizon in simulation steps. Within this window, we apply the knot parameterization of Eq. (3) in local time, using the corresponding segment of the reference commands and at most $K$ knots. For a candidate offset vector $z$, single shooting generates the predicted states $x_{t+1}(z), \dots, x_{t+H_t}(z)$ and evaluates

$$J_t(z) = \frac{1}{T}\left[\sum_{\tau=1}^{H_t} \ell(x_{t+\tau}(z); \bar{x}_{t+\tau}) + w_{\text{term}}\ell(x_{t+H_t}(z); \bar{x}_{t+H_t})\right]. \tag{18}$$

Here $J_t$ denotes the current window objective, and the normalization uses the full clip length $T$ at every replan. The terminal term encourages tracking at the window endpoint, where errors affect the next planning problem. Only a prefix, specified by the commit count $C$ in knots, is executed; the remaining controls can be revised in the next window. The horizon and executed prefix are shortened as needed at the end of the demonstration.

**Optimizing initial contact and motion together.** The initial hand configuration determines which contacts the first window can establish. For dexterous-hand retargeting, we optimize it jointly with the subsequent commands: the first control-offset knot also parameterizes the starting hand configuration,

$$q_0^{\text{hand}}(z) = \text{clip}_{[q_{\min}, q_{\max}]}(\bar{q}_0^{\text{hand}} + z_0), \qquad v_0^{\text{hand}} = 0. \tag{19}$$

Here $\bar{q}_0^{\text{hand}}$ is the reference hand configuration expressed in the same pose and joint coordinates as the position-target controls; $q_{\min}$ and $q_{\max}$ are componentwise bounds, and clip enforces those bounds. Each first-window candidate therefore starts from its own hand configuration, with zero hand velocity and objects at their reference state. Subsequent windows start from the robot and object states reached by execution. This couples initial contact selection to its motion consequences while preserving the simulated state across replans.

## 5.2 Reusing Plans and Particles Across Windows

Consecutive windows overlap, so the previous solution provides useful controls for much of the next window. We exploit this overlap through three complementary mechanisms.

**Warm-started priors.** We shift the previous window's selected plan by the executed prefix and use it to center the next window's search. The Gaussian AR(1) construction of Appendix A is applied to deviations from this shifted plan, so

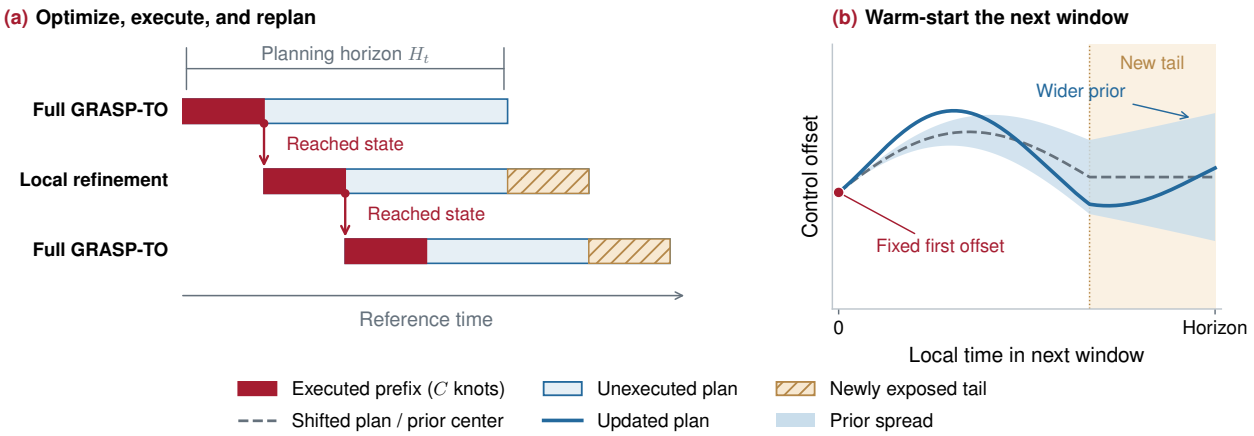


**Figure 4** GRASP-MPC. (a) Each window plans over a limited horizon and executes only a prefix (crimson), specified by the commit count $C$. The next window starts from the simulated state reached by that prefix. Full GRASP-TO searches alternate with local refinement, reusing overlapping plans while exposing a new tail. (b) For a full search, shifted unexecuted controls center the prior. The first offset is fixed to preserve command continuity, while the prior widens toward the tail to allow larger corrections. Timing, control values, and prior widths are illustrative.

samples are temporally correlated corrections to a promising continuation. The prior widens toward the window end, allowing more exploration where the previous plan supplies less information.

**Continuity at the window boundary.** After the first window, we pin the first knot to the offset carried over from the end of the executed prefix. Replanning therefore preserves the command at the shared boundary instead of introducing a jump. This constraint is enforced in both full searches and refinement windows; only the remaining offsets are optimized. It leaves the first window free to optimize its initial hand configuration through Eq. (19).

**Alternating full search and local refinement.** Full GRASP-TO searches alternate with cheaper refinement windows. A full search uses the shifted prior and applies the tempering, resampling, and surrogate-guided rejuvenation of Algorithm 1 to $J_t$. A refinement window instead shifts the previous window's candidate plans and re-evaluates them from the current state. These new evaluations are necessary because both the starting state and the reference segment have changed.

The refinement then samples a first generation of correlated local perturbations around promising shifted plans. A second generation selects parents from the retained candidates and the first-generation children, allowing improvements found in the first generation to guide further search. Both generations are scored with the current window objective. Refinement windows run no tempering stages: they reuse and improve existing candidates between full population searches.

### 5.3 Selecting and Executing a Plan

Before execution, we shortlist up to 50 distinct candidate plans and re-simulate them from the current state. We select the plan with the lowest mean finite rollout cost across its repeated evaluations. This additional comparison uses the same reference segment and objective for every candidate and reduces dependence on a single rollout evaluation.

We then execute only the committed prefix in the simulator and append its states and controls to the retargeted trajectory. The next window starts from the resulting state, and the selected plan and candidate population supply the next warm start or refinement. Repeating this cycle until the final reference frame produces a single dynamically consistent trajectory while adapting future commands to the contacts actually realized. Algorithm 2 summarizes this procedure.

## 6 Experiment: Dexterous Hands

We evaluate GRASP on dexterous manipulation, testing whether a common state-tracking objective can refine kinematically retargeted human demonstrations into physically executable robot motions. The experiments assess

**Algorithm 2** GRASP-MPC: closed-loop motion retargeting

**Require:** Initial state $x_0$, reference $\bar{x}_{0:T}$, window size $K$, commit count $C$
Set window index $j \leftarrow 0$, knot index $k \leftarrow 0$, library $\mathcal{L} \leftarrow \varnothing$
**while** $k < T/\Delta$ **do**
  Set $K_j \leftarrow \min(K, T/\Delta - k)$ and $C_j \leftarrow \min(C, K_j)$
  Form the window objective $J_j$ from the current simulated state using Eq. (18)
  **if** $j = 0$ **then**
    For dexterous hands, let each candidate's $z_0$ set its initial hand state by Eq. (19)
  **else**
    Shift the previous selected plan by $C_{j-1}$ knots and repeat its last knot to form the prior center
    Fix the first knot to the last executed control offset
  **end if**
  Construct the AR(1) window prior, allowing broader exploration toward the tail
  **if** $j$ is even **then**
    Run Algorithm 1 on $J_j$; retain its candidates and stage populations in $\mathcal{L}$
  **else**
    Shift $\mathcal{L}$ by $C_{j-1}$ knots, append an AR(1)-mean tail, and re-evaluate it under $J_j$
    Add two generations of correlated perturbations around promising candidates
  **end if**
  Re-evaluate up to 50 distinct leading candidates with repeated rollouts
  Select $z^\star$ with the lowest mean finite rollout cost
  Execute its first $C_j$ knots; append the resulting states and controls
  Set the current state to the reached state; store $z^\star$ and set $k \leftarrow k + C_j$, $j \leftarrow j + 1$
**end while**
**return** The executed state and control trajectory

retargeting quality on large demonstration collections, transfer across hand embodiments, and the value of the resulting trajectories for policy learning and real-robot execution.

**Implementation.** We use GRASP-MPC with MuJoCo [39] and its Warp backend[1]. All dexterous-hand retargeting experiments share the tracking objective in Eq. (2) and the same optimizer settings. The objective tracks robot and object states without a contact reward, leaving the optimizer to discover contacts through simulated interactions under gravity. We jointly optimize the initial hand configurations and the first control window to refine the starting grasp. Appendix C gives the implementation details, task-selection protocol, and metric definitions.

## 6.1 Bimanual Retargeting on OakInk2

**Setup.** We evaluate on 1352 bimanual demonstrations from OakInk2 [45], retargeted to two Sharpa hands. The collection includes cutting, pouring, stirring, uncapping, inserting, and opening, with a mean duration of 8.16 s. These tasks require coordinated contacts between both hands and the manipulated objects while preserving the demonstrated object motion. Following the object-tracking criterion used by Do-as-I-Do [26], a trajectory succeeds when its mean object position error is below 10 cm and its mean rotation error is below 0.5 rad. Runs that terminate before completing the trajectory count as failures. Table 1 provides published reference results for kinematic dex-retargeting, SPIDER, and Do-as-I-Do, reproduced from Wang et al. [40].

**Results.** GRASP succeeds on 1274/1352 demonstrations (94.2%). Mean object position and rotation errors are 2.87 cm and 0.117 rad, with medians of 2.17 cm and 0.090 rad. These mean errors are well below the success thresholds. Published reference success rates range from 28.6% for kinematic retargeting to 81.0% for Do-as-I-Do (Table 1). On our evaluation collection, GRASP recovers bimanual interactions across diverse tasks using a shared tracking objective without task-specific contact guidance.

[1] https://github.com/google-deepmind/mujoco_warp.

**Table 1** Bimanual retargeting with Sharpa hands. GRASP is evaluated on 1352 OakInk2 demonstrations. Published reference results are reproduced from Wang et al. [40], Table 2 (OakInk). $E_{\text{pos}}$ and $E_{\text{rot}}$ denote mean object position and rotation errors.

| Method | Succ. (%) ↑ | $E_{\text{pos}}$ (m) ↓ | $E_{\text{rot}}$ (rad) ↓ |
|---|---|---|---|
| Dex-retargeting | 28.6 | 0.08 | 0.62 |
| SPIDER mjwp | 71.4 | 0.04 | 0.57 |
| SPIDER mjwp_act | 77.1 | 0.04 | 0.42 |
| Do-as-I-Do (Sharpa) | 81.0 | 0.03 | 0.15 |
| w/o transition reward | 79.0 | 0.03 | 0.14 |
| annealed sampling only | 72.0 | 0.08 | 0.32 |
| **GRASP** | **94.2** | **0.03** | **0.12** |

## 6.2 Cross-Embodiment Retargeting

**Setup.** We next test whether the same configuration transfers to hands with different kinematics. We apply GRASP-MPC to OakInk2 demonstrations using bimanual Wuji and XHand models, which have 20 and 12 finger joints per hand, respectively, compared with 22 for Sharpa. Each hand also has a 6-DoF base. We change the robot model and its kinematic reference while keeping the tracking objective and optimizer settings fixed. Table 2 reports results on the same 1,352-task collection for each embodiment.

**Table 2** Cross-embodiment retargeting on OakInk2 with one optimizer configuration on 1,352 tasks per hand. Execution failures count as failures in the success rate. Finger DoF are per hand; every hand also has a 6-DoF base. $E_{\text{pos}}$ and $E_{\text{rot}}$ are mean / median object position and rotation errors over completed trajectories with finite metrics.

| Hand | Finger DoF | Tasks | Succ. (%) ↑ | $E_{\text{pos}}$ (cm) ↓ | $E_{\text{rot}}$ (rad) ↓ |
|---|---|---|---|---|---|
| Sharpa | 22 | 1352 | 94.2 | 2.87 / 2.17 | 0.117 / 0.090 |
| Wuji | 20 | 1352 | 92.8 | 2.87 / 2.02 | 0.117 / 0.086 |
| XHand | 12 | 1352 | 95.0 | 2.87 / 2.08 | 0.112 / 0.088 |

**Results.** On the evaluated clips, success rates exceed 92% for all three hands: 92.8% for Wuji, 95.0% for XHand, and 94.2% for Sharpa. Wuji and XHand both achieve a mean object position error of 2.87 cm, with mean rotation errors of 0.117 and 0.112 rad, respectively. These results support the use of a shared tracking formulation and optimizer configuration across the three tested embodiments, including the 12-joint XHand.

**Manipulation across embodiments.** Figure 5 shows three representative interactions: Sharpa hands wipe a whiteboard, Wuji hands bring a pen and cap together, and XHand hands stir in a cup. The sequences require sustained surface contact, alignment of grasped parts, and coordinated tool–container motion, respectively, illustrating the range of interactions supported by the shared configuration.

## 6.3 Retargeting HOT3D Demonstrations

**Setup.** To assess performance beyond OakInk2, we retarget 5,400 HOT3D trajectories [2] from the Aria and Quest3 recordings to Sharpa hands. The collection includes right-hand, left-hand, and bimanual interactions. We measure object tracking with ADD-AUC, which integrates the fraction of frames satisfying an object-surface alignment threshold as the threshold varies from 0 to 10 cm. This metric captures translation and rotation in a single score without fixing one threshold. Table 3 includes the right-hand results reported by SPIDER [27] alongside our breakdown by interaction type; each row identifies its evaluation set.

**Results.** GRASP achieves 81.54% ADD-AUC over the full collection. Right- and left-hand trajectories attain similar scores (84.87% and 85.44%), while bimanual trajectories reach 75.83%, the lowest score among the three interaction types in this collection. A single configuration thus supports thousands of demonstrations from both recording platforms, including one- and two-hand interactions. SPIDER reports 40.74% for Sharpa on its right-hand evaluation set.

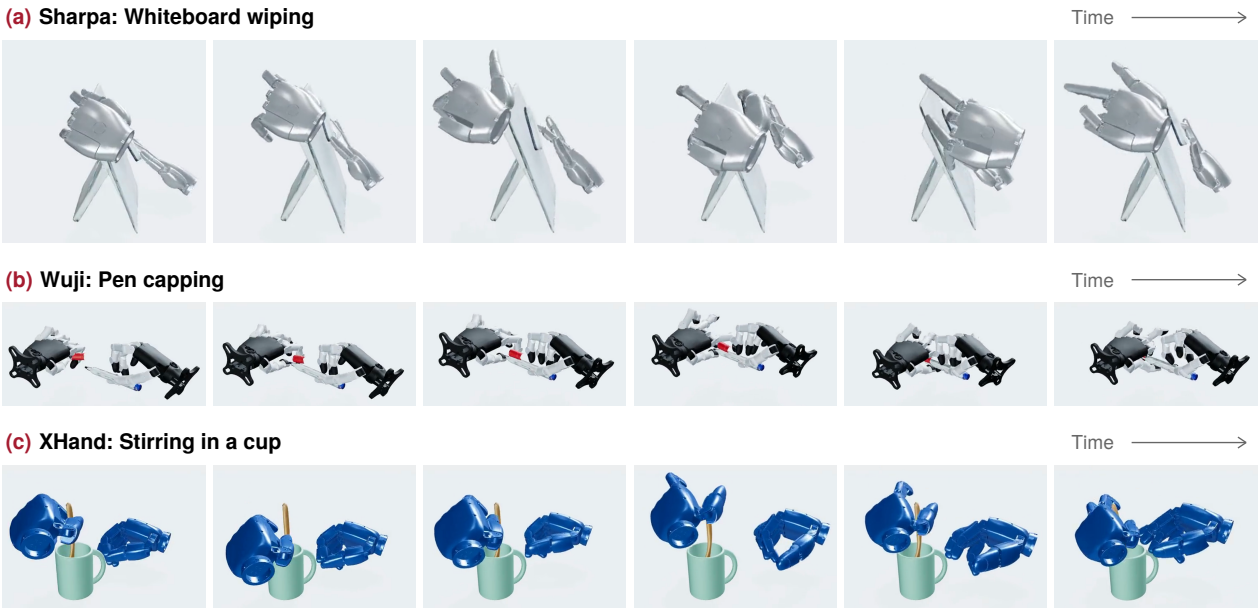


**Figure 5** Simulated dexterous manipulation across three hand embodiments. Rows show (a) Sharpa wiping a whiteboard, (b) Wuji capping a pen, and (c) XHand stirring in a cup. Each row shows six frames from one retargeted OakInk2 trajectory, ordered from left to right, with a fixed crop.

**Table 3** Object tracking on HOT3D. ADD-AUC integrates the fraction of frames whose mean object-surface point error falls below a threshold over thresholds of 0–10 cm, normalized to a percentage. We use 512 area-uniform surface samples and average trajectories equally. SPIDER results are reproduced from its Table 3; GRASP uses Sharpa hands on the evaluated Aria and Quest3 clips. The methods use different clip collections.

| | | | | ADD-AUC (%) ↑ |
|---|---|---|---|---|
| Method | Split | # Demos | # Objects | Sharpa |
| SPIDER [27] | Right hand | 2,718 | 33 | 40.74 |
| **GRASP** | All | 5,400 | 33 | **81.54** |
| | Right hand | 2,117 | 33 | 84.87 |
| | Left hand | 1,213 | 33 | 85.44 |
| | Bimanual | 2,070 | 33 | 75.83 |

## 6.4 Policy Learning from Retargeted Trajectories

**Setup.** We assess the downstream value of reference refinement through single-axis pen spinning. On a group of nine clips, we train one PPO policy to track the original kinematic references and one to track their GRASP-optimized counterparts. Both conditions use the same robot, reward configuration, domain randomization, and training budget, varying only the reference. Figure 6 reports episode reward, episode length, and object tracking errors during training.

**Results.** Over the final 500 training iterations, the policy trained on GRASP references achieves a mean episode reward of 45.9, compared with 21.7 for the kinematic references. Mean episode length increases from 151 to 235 steps, and position error decreases from 1.98 to 0.99 cm. Shaft-angle error increases slightly, from 3.9 to 4.3 deg. The gains therefore lie in longer tracking episodes and lower object position error, with a small tradeoff in shaft-angle accuracy. The learning curves show these benefits over much of training, supporting the use of physics-refined references for this task.

## 6.5 Real-Robot Execution

We execute GRASP-generated trajectories on physical dexterous hands through open-loop playback. Figure 7 shows two manipulation sequences: lifting a pumpkin-shaped container by its handle and pouring from a can into a cup. The first requires supporting an object through a narrow handle; the second requires coordinating the container

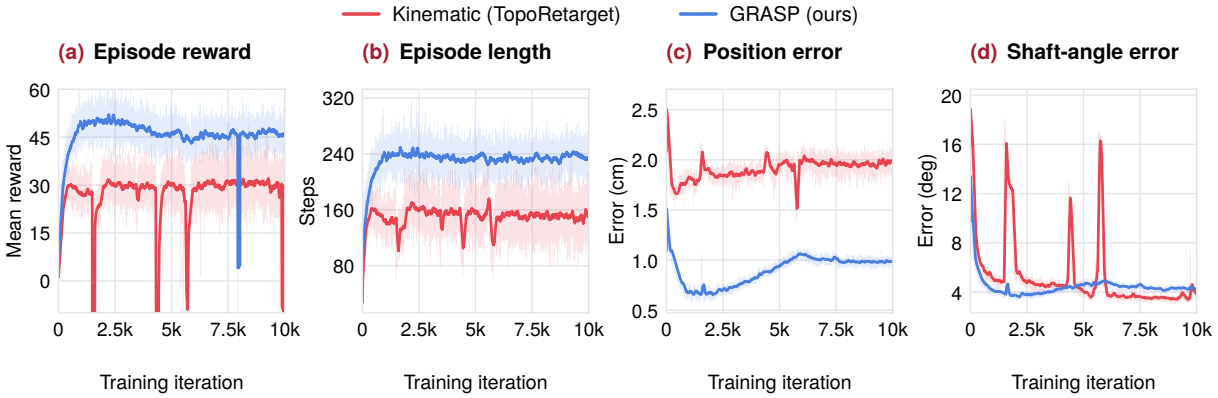


**Figure 6** PPO learning on nine single-axis pen-spinning clips using kinematic references (TopoRetarget) or GRASP-refined references (ours). Panels show (a) episode reward, (b) episode length, (c) object position error, and (d) shaft-angle error. Bold curves are 100-iteration moving averages of the training logs; faint curves are the unsmoothed values.

orientations during pouring. These demonstrations show physical execution of the optimized motions for two distinct grasp geometries and object interactions. Additional demonstrations are available on the project website.

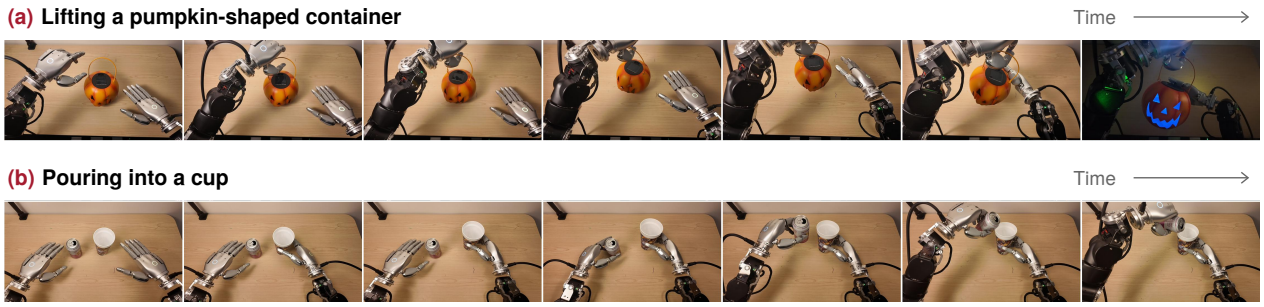


**Figure 7** Open-loop trajectory playback with real dexterous hands. (a) Lifting a pumpkin-shaped container by its handle. (b) Pouring from a can into a cup. Frames progress from left to right.

# 7 Experiment: Humanoids

We next evaluate humanoid motion, which requires coordinating whole-body movement, ground contact, and object manipulation. We measure retargeting accuracy on the Unitree G1, illustrate interactions across three humanoid embodiments, and assess the effect of reference refinement on policy learning. Real-robot demonstrations examine crawling and box manipulation.

## 7.1 Whole-Body Motion Retargeting

**Setup.** We retarget motions from LAFAN1 [12], AMASS [23], and OMOMO [20] to the Unitree G1 using GRASP-MPC. LAFAN1 and AMASS evaluate robot motion tracking, while OMOMO also includes box manipulation. We report joint and pelvis tracking errors on all three datasets and object pose errors on OMOMO. Table 4 includes the GMR [1] and SPIDER results reported by Pan et al. [27]. Appendix D describes the evaluation and optimizer configuration.

**Results.** Table 4 summarizes robot tracking on all three datasets and joint robot–object tracking on OMOMO. For OMOMO, the recorded GRASP errors are 0.21 for the joints, 0.06 for pelvis translation, and 0.03 for object translation.

**Interactions across embodiments.** Figure 8 illustrates object manipulation with three humanoid models. The Unitree G1 lifts a box from the floor, the Unitree H2 lifts and turns a table, and the Booster T1 raises a chair overhead. These sequences require coordinated reaching, object support, and changes in body posture. They illustrate the range of whole-body interactions produced by the formulation beyond the G1 quantitative evaluation.

**Table 4** Whole-body tracking on the Unitree G1. $e_j$ denotes joint error; $e_t$ and $e_r$ denote translation and orientation errors for the pelvis and, on OMOMO, the object. GMR and SPIDER values are reproduced from Pan et al. [27], Table 2.

| | | | Robot | | | Object | |
|---|---|---|---|---|---|---|---|
| Robot | Data | Method | $e_j \downarrow$ | $e_t \downarrow$ | $e_r \downarrow$ | $e_t \downarrow$ | $e_r \downarrow$ |
| G1 | LAFAN1 | GMR | 1.08 | 2.01 | 2.40 | – | – |
| | | SPIDER | 0.58 | 0.11 | 0.07 | – | – |
| | | **GRASP** | 0.30 | 0.02 | 0.06 | – | – |
| G1 | AMASS | GMR | 6.2 | 4.1 | 18.7 | – | – |
| | | SPIDER | 0.75 | 0.23 | 0.08 | – | – |
| | | **GRASP** | 0.59 | 0.05 | 0.12 | – | – |
| G1 | OMOMO | GMR | 0.57 | 1.43 | 2.28 | 1.18 | 1.68 |
| | | SPIDER | 0.83 | 0.20 | 0.17 | 0.18 | 0.06 |
| | | **GRASP** | 0.21 | 0.06 | 0.06 | 0.03 | 0.13 |

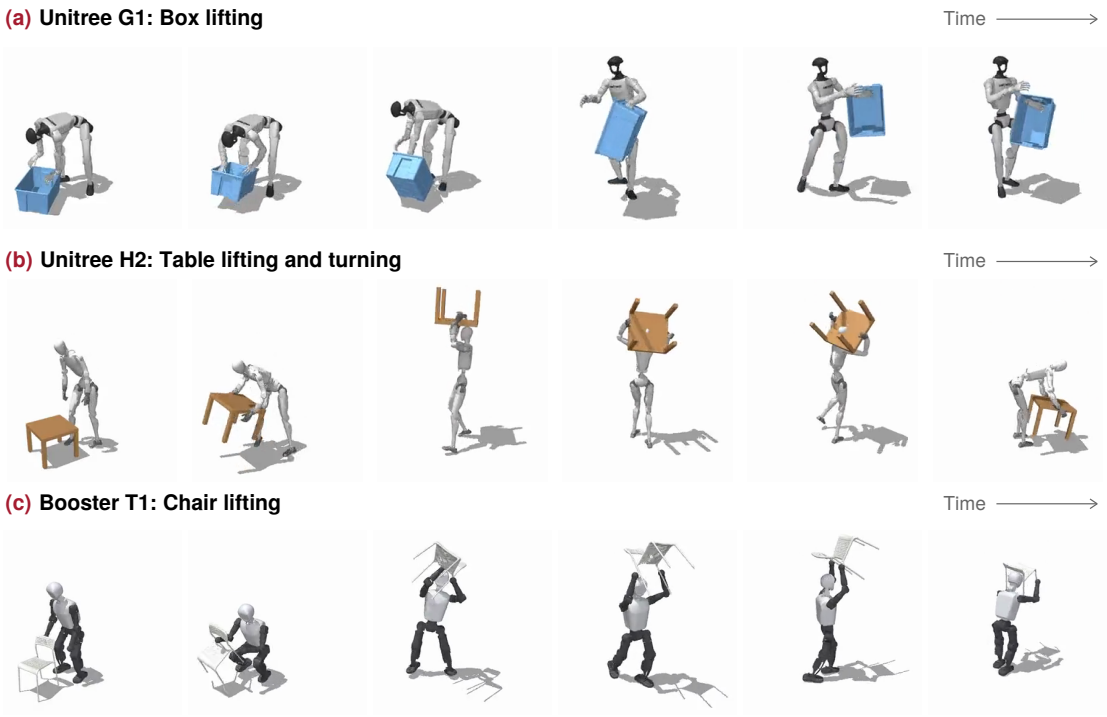


**Figure 8** Simulated humanoid–object interactions generated by GRASP. Rows show (a) the Unitree G1 lifting a box, (b) the Unitree H2 lifting and turning a table, and (c) the Booster T1 lifting a chair. Each row shows six frames from one trajectory, ordered from left to right.

## 7.2 Learning Humanoid Policies from Refined References

**Setup.** Following the downstream RL setup of DynaRetarget [8], we train PPO tracking policies for the Unitree G1 using three reference motions from OmniRetarget [43]. For each motion, we compare policies trained on the original kinematic trajectory with policies trained on its GRASP-refined counterpart. Both conditions use the same PPO configuration, reward weights, and domain randomization. We report object position and orientation rewards, each measured relative to the corresponding training reference, to assess how refinement affects learning to track robot–object interactions.

**Results.** Figure 9 shows that GRASP references yield higher mean object position and orientation rewards through most of training. The curves separate early and remain separated at the end of the 30,000-iteration budget. This pattern suggests that the refined motions provide more learnable tracking references under the shared training configuration.

Together with the pen-spinning results, these experiments support the use of physics-based refinement for downstream policy learning in both dexterous-hand and humanoid tasks.

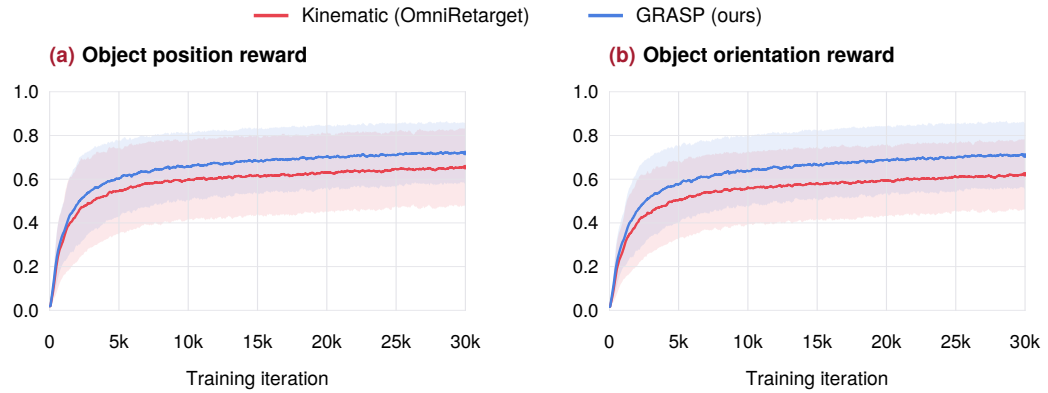


**Figure 9** Humanoid PPO learning with three reference motions from OmniRetarget. (a) Object position and (b) orientation tracking rewards are compared for the original kinematic trajectories (red) and GRASP-refined trajectories (blue). Higher rewards indicate better tracking of the corresponding reference.

## 7.3 Real-Robot Execution

We deploy tracking policies trained on GRASP-refined references to control real humanoids in closed loop; deployment follows OmniRetarget [43]. Figure 10 shows crawling, box carrying, and box pushing. Crawling requires sustained limb–floor contact. During box carrying, the robot bends down, lifts the box, walks while supporting it, and returns it to the floor. During box pushing, the robot reaches down and moves the box along the floor before releasing it.

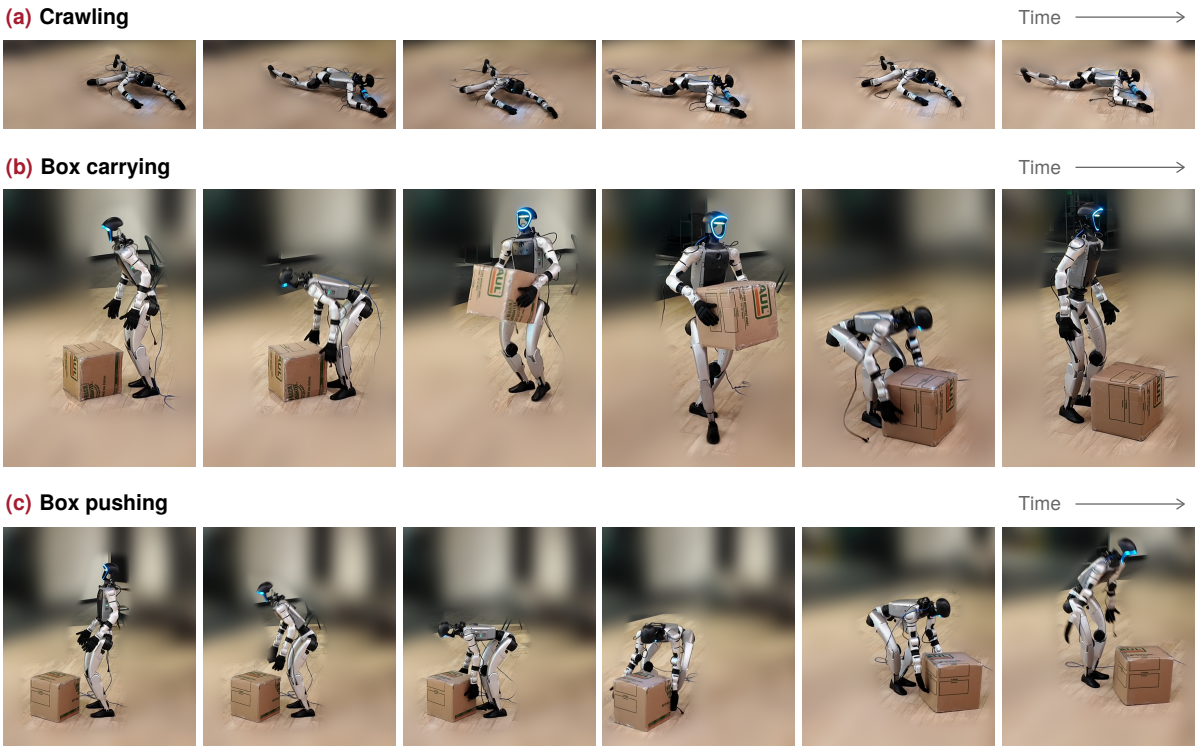


**Figure 10** Closed-loop execution of learned policies on real humanoids. Rows show (a) crawling, (b) lifting, carrying, and placing a box, and (c) pushing a box along the floor. Each row shows six frames from one execution, ordered from left to right. Backgrounds are blurred for clarity, preserving the robot, manipulated object, and immediate contact regions.

These demonstrations cover ground contact, object support during locomotion, and object displacement through hand contact.

# 8 Conclusion

We presented GRASP, a method that combines sampling-based exploration and gradient-guided refinement for physics-based motion retargeting without differentiable simulator dynamics. GRASP-TO recovers guidance from local quadratic models of archived rollouts, with a Metropolis correction that preserves the original target for fixed proposal parameters. Our analysis relates the population fit to a Gaussian-smoothed cost, while GRASP-MPC extends the solver to long demonstrations by reusing plans across overlapping control windows.

Experiments show that a shared state-tracking objective can recover contacts across dexterous-hand and humanoid demonstrations without task-specific contact rewards. The refined references improve downstream PPO tracking, and hardware demonstrations include dexterous manipulation, humanoid crawling, and box manipulation. These results support rollout reuse as a practical source of optimization guidance and training references for robot learning. Future work will study the reliability of guidance from finite, regularized archives and the robustness of the resulting motions and policies to errors in object geometry and dynamics.

# References


[1] Joao Pedro Araujo, Yanjie Ze, Pei Xu, Jiajun Wu, and C. Karen Liu. Retargeting matters: General motion retargeting for humanoid motion tracking. *arXiv preprint arXiv:2510.02252*, 2025. URL https://arxiv.org/abs/2510.02252.

[2] Prithviraj Banerjee, Sindi Shkodrani, Pierre Moulon, Shreyas Hampali, Shangchen Han, Fan Zhang, Linguang Zhang, Jade Fountain, Edward Miller, Selen Basol, Richard Newcombe, Robert Wang, Jakob Julian Engel, and Tomas Hodan. HOT3D: Hand and object tracking in 3D from egocentric multi-view videos. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025. URL https://arxiv.org/abs/2411.19167.

[3] Alexandros Beskos, Ajay Jasra, Nikolas Kantas, and Alexandre Thiery. On the convergence of adaptive sequential Monte Carlo methods. *The Annals of Applied Probability*, 26(2):1111–1146, 2016. doi: 10.1214/15-AAP1113. URL https://arxiv.org/abs/1306.6462.

[4] Homanga Bharadhwaj, Kevin Xie, and Florian Shkurti. Model-predictive control via cross-entropy and gradient-based optimization. In *Proceedings of the 2nd Conference on Learning for Dynamics and Control*, volume 120 of *Proceedings of Machine Learning Research*, pages 277–286, 2020. URL https://proceedings.mlr.press/v120/bharadhwaj20a.html.

[5] Xuxin Cheng, Yandong Ji, Junming Chen, Ruihan Yang, Ge Yang, and Xiaolong Wang. Expressive whole-body control for humanoid robots. *arXiv preprint arXiv:2402.16796*, 2024. URL https://arxiv.org/abs/2402.16796.

[6] Andrew R. Conn, Katya Scheinberg, and Luis N. Vicente. *Introduction to Derivative-Free Optimization*. SIAM, 2009. doi: 10.1137/1.9780898718768.

[7] Pierre Del Moral, Arnaud Doucet, and Ajay Jasra. Sequential Monte Carlo samplers. *Journal of the Royal Statistical Society: Series B*, 68(3):411–436, 2006. URL https://www.stats.ox.ac.uk/~doucet/delmoral_doucet_jasra_sequentialmontecarlosamplersJRSSB.pdf.

[8] Victor Dhédin, Ilyass Taouil, Shafeef Omar, Dian Yu, Kun Tao, Angela Dai, and Majid Khadiv. DynaRetarget: Dynamically-feasible retargeting using sampling-based trajectory optimization. *arXiv preprint arXiv:2602.06827*, 2026. URL https://arxiv.org/abs/2602.06827.

[9] Sergio A. Esteban, Jason H. K. Siu, Derrick Mach, Junheng Li, Vince Kurtz, Joel W. Burdick, and Aaron D. Ames. Shooting for contact: Contact-implicit multiple shooting for dynamic motion retargeting. *arXiv preprint arXiv:2608.03116*, 2026. URL https://arxiv.org/abs/2608.03116.

[10] Bernhard Paus Graesdal, Shao Yuan Chew Chia, Tobia Marcucci, Savva Morozov, Alexandre Amice, Pablo A. Parrilo, and Russ Tedrake. Towards tight convex relaxations for contact-rich manipulation. In *Robotics: Science and Systems*, 2024. doi: 10.15607/RSS.2024.XX.132. URL https://arxiv.org/abs/2402.10312.

[11] Antoine Groudiev, Shucheng Kang, and Heng Yang. cuADMM: GPU-accelerated first-order optimization for large-scale multi-block semidefinite programs. In *RSS Workshop on Fast Motion Planning and Control in the Era of Parallelism*, 2025. URL https://agroudiev.github.io/assets/pdf/8_cuADMM_GPU_Accelerated_First.pdf.

[12] Félix G. Harvey, Mike Yurick, Derek Nowrouzezahrai, and Christopher Pal. Robust motion in-betweening. *ACM Transactions on Graphics (SIGGRAPH)*, 2020. URL https://arxiv.org/abs/2102.04942.

[13] W. K. Hastings. Monte carlo sampling methods using markov chains and their applications. *Biometrika*, 57(1):97–109, 1970. doi: 10.1093/biomet/57.1.97.

[14] Tairan He, Zhengyi Luo, Wenli Xiao, Chong Zhang, Kris Kitani, Changliu Liu, and Guanya Shi. Learning human-to-humanoid real-time whole-body teleoperation. *arXiv preprint arXiv:2403.04436*, 2024. URL https://arxiv.org/abs/2403.04436.

[15] Tairan He, Jiawei Gao, Wenli Xiao, Yuanhang Zhang, Zi Wang, Jiashun Wang, Zhengyi Luo, Guanqi He, Nikhil Sobanbab, Chaoyi Pan, Zeji Yi, Guannan Qu, Kris Kitani, Jessica Hodgins, Linxi Fan, Yuke Zhu, Changliu Liu, and Guanya Shi. ASAP: Aligning simulation and real-world physics for learning agile humanoid whole-body skills. In *Robotics: Science and Systems (RSS)*, 2025. URL https://arxiv.org/abs/2502.01143.

[16] Kohei Honda, Naoki Akai, Kosuke Suzuki, Mizuho Aoki, Hirotaka Hosogaya, Hiroyuki Okuda, and Tatsuya Suzuki. Stein variational guided model predictive path integral control: Proposal and experiments with fast maneuvering vehicles. *arXiv preprint arXiv:2309.11040*, 2023. URL https://arxiv.org/abs/2309.11040.

[17] Mrinal Kalakrishnan, Sachin Chitta, Evangelos Theodorou, Peter Pastor, and Stefan Schaal. STOMP: Stochastic trajectory optimization for motion planning. In *IEEE International Conference on Robotics and Automation*, 2011. URL https://is.mpg.de/publications/kalakrishnan_raiic_2011.

[18] Shucheng Kang, Guorui Liu, and Heng Yang. Global contact-rich planning with sparsity-rich semidefinite relaxations. In *Robotics: Science and Systems*, 2025. doi: 10.15607/RSS.2025.XXI.046. URL https://arxiv.org/abs/2502.02829.

[19] Shucheng Kang, Xiaoyang Xu, Jay Sarva, Ling Liang, and Heng Yang. Fast and certifiable trajectory optimization. In *Algorithmic Foundations of Robotics XVI, Volume 1*, volume 37 of *Springer Proceedings in Advanced Robotics*, pages 43–65. Springer, 2026. doi: 10.1007/978-3-032-09967-9_3. URL https://link.springer.com/chapter/10.1007/978-3-032-09967-9_3.

[20] Jiaman Li, Jiajun Wu, and C. Karen Liu. Object motion guided human motion synthesis. *ACM Transactions on Graphics (SIGGRAPH Asia)*, 2023. URL https://arxiv.org/abs/2309.16237.

[21] Jiayun Li, Dejian Gong, and Georgia Chalvatzaki. IMPACT: An implicit active-set augmented lagrangian for fast contact-implicit trajectory optimization. In *Robotics: Science and Systems*, 2026. doi: 10.15607/RSS.2026.XXII.163.

[22] Yulin Li, Haoyu Han, Shucheng Kang, Jun Ma, and Heng Yang. On the surprising robustness of sequential convex optimization for contact-implicit motion planning. In *Robotics: Science and Systems*, 2025. doi: 10.15607/RSS.2025.XXI.047.

[23] Naureen Mahmood, Nima Ghorbani, Nikolaus F. Troje, Gerard Pons-Moll, and Michael J. Black. AMASS: Archive of motion capture as surface shapes. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019. URL https://arxiv.org/abs/1904.03278.

[24] Igor Mordatch, Zoran Popovic, and Emanuel Todorov. Contact-invariant optimization for hand manipulation. In *Eurographics/ACM SIGGRAPH Symposium on Computer Animation*, pages 137–144, 2012. doi: 10.2312/SCA/SCA12/137-144.

[25] Yurii Nesterov and Vladimir Spokoiny. Random gradient-free minimization of convex functions. *Foundations of Computational Mathematics*, 17(2):527–566, 2017. doi: 10.1007/s10208-015-9296-2.

[26] Bhawna Paliwal, Haritheja Etukuru, William Liang, Pieter Abbeel, Nur Muhammad Mahi Shafiullah, and Jitendra Malik. Do as i do: Dexterous manipulation data from everyday human videos. *arXiv preprint arXiv:2606.19333*, 2026. URL https://arxiv.org/abs/2606.19333.

[27] Chaoyi Pan, Changhao Wang, Haozhi Qi, Julen Urain, Zixi Liu, Homanga Bharadhwaj, Akash Sharma, Tingfan Wu, Guanya Shi, Jitendra Malik, and Francois Hogan. SPIDER: Scalable physics-informed dexterous retargeting. *arXiv preprint arXiv:2511.09484*, 2025. URL https://arxiv.org/abs/2511.09484.

[28] Kisang Park, Chan Woo Kim, Kyungjae Lee, and Sungjoon Choi. Natural functional gradients for smooth trajectory optimization. In *Robotics: Science and Systems*, 2026. doi: 10.15607/RSS.2026.XXII.162.

[29] Cristina Pinneri, Shambhuraj Sawant, Sebastian Blaes, Jan Achterhold, Joerg Stueckler, Michal Rolinek, and Georg Martius. Sample-efficient cross-entropy method for real-time planning. In *Proceedings of the 2020 Conference on Robot Learning*, volume 155 of *Proceedings of Machine Learning Research*, pages 1049–1065, 2021. URL https://proceedings.mlr.press/v155/pinneri21a.html.

[30] Michael Posa, Cecilia Cantu, and Russ Tedrake. A direct method for trajectory optimization of rigid bodies through contact. *The International Journal of Robotics Research*, 33(1):69–81, 2014. doi: 10.1177/0278364913506757.

[31] Yuzhe Qin, Yueh-Hua Wu, Shaowei Liu, Hanwen Jiang, Ruihan Yang, Yang Fu, and Xiaolong Wang. DexMV: Imitation learning for dexterous manipulation from human videos. *arXiv preprint arXiv:2108.05877*, 2021. URL https://arxiv.org/abs/2108.05877.

[32] Daniel Rakita, Chen Liang, and Qian Wang. Coherence-based approximate derivatives via web of affine spaces optimization. In *Robotics: Science and Systems*, 2025. doi: 10.15607/RSS.2025.XXI.024.

[33] Gareth O. Roberts and Richard L. Tweedie. Exponential convergence of langevin distributions and their discrete approximations. *Bernoulli*, 2(4):341–363, 1996. doi: 10.2307/3318418.

[34] Reuven Rubinstein. The cross-entropy method for combinatorial and continuous optimization. *Methodology and Computing in Applied Probability*, 1(2):127–190, 1999. doi: 10.1023/A:1010091220143.

[35] H. J. Terry Suh, Tao Pang, and Russ Tedrake. Bundled gradients through contact via randomized smoothing. *IEEE Robotics and Automation Letters*, 2022. URL https://arxiv.org/abs/2109.05143.

[36] H. J. Terry Suh, Tao Pang, Tong Zhao, and Russ Tedrake. Dexterous contact-rich manipulation via the contact trust region. *The International Journal of Robotics Research*, 45(9):1418–1454, 2026. doi: 10.1177/02783649251398875.

[37] Hyung Ju Suh, Max Simchowitz, Kaiqing Zhang, and Russ Tedrake. Do differentiable simulators give better policy gradients? In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 20668–20696, 2022. URL https://proceedings.mlr.press/v162/suh22b.html.

[38] Xudong Sun, Armand Jordana, Massimo Fornasier, Jalal Etesami, and Majid Khadiv. Consensus-based optimization (CBO): Towards global optimality in robotics. In *Robotics: Science and Systems*, 2026. doi: 10.15607/RSS.2026.XXII.161.

[39] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033, 2012. doi: 10.1109/IROS.2012.6386109.

[40] Youhui Wang, Yunzhu Li, Li Fei-Fei, Jiajun Wu, and Huang Huang. DexAgent: An agentic human2sim2robot framework for dexterous manipulation with self-evolving tool library. *arXiv preprint arXiv:2609.35318*, 2026.

[41] Grady Williams, Nolan Wagener, Brian Goldfain, Paul Drews, James M. Rehg, Byron Boots, and Evangelos A. Theodorou. Information theoretic MPC for model-based reinforcement learning. In *IEEE International Conference on Robotics and Automation*, 2017. doi: 10.1109/ICRA.2017.7989202. URL https://homes.cs.washington.edu/~bboots/files/InformationTheoreticMPC.pdf.

[42] Heng Yang. Tempered sequential monte carlo for trajectory and policy optimization with differentiable dynamics. In *Robotics: Science and Systems*, 2026. doi: 10.15607/RSS.2026.XXII.109. URL https://arxiv.org/abs/2604.21456.

[43] Lujie Yang, Xiaoyu Huang, Zhen Wu, Angjoo Kanazawa, Pieter Abbeel, Carmelo Sferrazza, C. Karen Liu, Rocky Duan, and Guanya Shi. OmniRetarget: Interaction-preserving data generation for humanoid whole-body loco-manipulation and scene interaction. *arXiv preprint arXiv:2509.26633*, 2025. URL https://arxiv.org/abs/2509.26633.

[44] Lujie Yang, H. J. Terry Suh, Tong Zhao, Bernhard Paus Graesdal, Tarik Kelestemur, Jiuguang Wang, Tao Pang, and Russ Tedrake. Physics-driven data generation for contact-rich manipulation via trajectory optimization. In *Robotics: Science and Systems*, 2025. doi: 10.15607/RSS.2025.XXI.053.

[45] Xinyu Zhan, Lixin Yang, Yifei Zhao, Kangrui Mao, Hanlin Xu, Zenan Lin, Kailin Li, and Cewu Lu. OAKINK2: A dataset of bimanual hands-object manipulation in complex task completion. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024. URL https://arxiv.org/abs/2403.19417.

# Appendix

## A AR(1) Prior

The Gaussian autoregressive prior $p_0(z)$ correlates successive knot offsets within each control channel while keeping channels independent. For channel $c \in \{1, \dots, m\}$ in Eq. (3), we sample

$$z_{0,c} \sim \mathcal{N}\Big(0, \tfrac{s_{0,c}^2}{1-\rho^2}\Big), \qquad z_{k,c} = \rho\, z_{k-1,c} + s_{k,c}\, \xi_{k,c}, \quad \xi_{k,c} \sim \mathcal{N}(0,1), \tag{20}$$

for $k = 1, \dots, K-1$, with independent standard-normal perturbations $\xi_{k,c}$. Here $\rho \in [0, 1)$ controls temporal correlation, and $s_{k,c} = \sigma\kappa_{k,c}$ is the exploration scale, with base scale $\sigma > 0$ and channel- and knot-dependent factors $\kappa_{k,c} > 0$. The corresponding negative log density is

$$-\log p_0(z) = \frac{1}{2} \sum_{c=1}^{m} \Big[ \frac{(1-\rho^2)\, z_{0,c}^2}{s_{0,c}^2} + \sum_{k=1}^{K-1} \frac{(z_{k,c} - \rho z_{k-1,c})^2}{s_{k,c}^2} \Big] + \text{const.} \tag{21}$$

With knot-independent scales $s_{k,c} = s_c$, the initialization in Eq. (20) makes the process stationary, with variance $s_c^2/(1-\rho^2)$ and correlation $\rho^{|k-k'|}$ between knots $k$ and $k'$. In the dexterous-hand experiments, we use $\rho = 0.9$, $\sigma = 0.0044$, and a factor of 10 on finger channels. For MPC windows with $K > 1$ knots, we additionally multiply the scales by $4^{k/(K-1)}$ to increase exploration toward the window end. As described in Section 5.2, the MPC prior is centered on the shifted previous plan, and the boundary knot is fixed after the first window.

**Interpretation.** Each knot retains a fraction $\rho$ of the preceding offset and adds a perturbation with standard deviation $s_{k,c}$. The stationary prior therefore favors sustained control corrections: with $\rho = 0.9$ and 0.2 s knot spacing, offsets one second apart have correlation $0.9^5 \approx 0.59$. Before window-dependent widening or boundary conditioning, the nominal stationary standard deviations are about 1 cm for hand-base positions, 0.01 rad for base rotations, and 0.1 rad for finger joints. The larger finger scale allows more variation in grasp configuration, while the base remains closer to the reference. Widening the MPC prior provides additional freedom where the shifted plan supplies less information. The prior covariance also preconditions the Langevin proposals, giving their perturbations the same temporal structure.

## B Gradient Recovery and Metropolis Correction

### B.1 Derivatives of a Smoothed Objective

We first justify replacing full rollout costs with the reduced response in Eq. (14). Let $Z \sim \nu_z$, $Y = B^\top D^{-1}(Z - \mu)$, and $F(Y) = \mathbb{E}[J(Z) \mid Y]$. Assume $\mathbb{E}[J(Z)^2] < \infty$. For any square-integrable predictor $h(Y)$, conditioning on $Y$ gives

$$\mathbb{E}[(J(Z) - h(Y))^2] = \mathbb{E}[\mathrm{Var}(J(Z) \mid Y)] + \mathbb{E}[(F(Y) - h(Y))^2]. \tag{22}$$

The conditional-variance term is independent of $h$. Consequently, fitting a quadratic to $J(Z)$ or to $F(Y)$ yields the same population minimizer.

We prove Proposition 1 under its stated Gaussian fitting-measure and polynomial-growth assumptions.

*Proof.* Express the quadratic in the centered Gaussian polynomial basis as $a^\circ + b^\top Y + \frac{1}{2}\,\mathrm{tr}[A(YY^\top - \Sigma)]$, where $a^\circ = a + \frac{1}{2}\,\mathrm{tr}(A\Sigma)$. The constant, linear, and centered quadratic blocks are orthogonal in $L^2(\nu)$. Their normal equations give

$$b_* = \Sigma^{-1}\mathbb{E}[YF(Y)], \tag{23}$$

$$A_* = \Sigma^{-1}\mathbb{E}[(YY^\top - \Sigma)F(Y)]\Sigma^{-1}. \tag{24}$$

The remaining coefficient is $a_* = \mathbb{E}[F(Y)] - \frac{1}{2}\,\mathrm{tr}(A_*\Sigma)$. Since $\Sigma \succ 0$, these normal equations determine a unique quadratic. Differentiating the Gaussian convolution kernel yields the same expressions for $\nabla\overline{F}(0)$ and $\nabla^2\overline{F}(0)$, establishing Eq. (17). Polynomial growth ensures integrability and permits differentiation under the integral; no derivative of $F$ is required. □

### B.2 Fixed-Kernel Metropolis Correction

The surrogate determines where the Markov kernel proposes to move, while the acceptance test uses the original rollout objective. The following statement formalizes why fitting error does not change the invariant distribution of a fixed kernel.

**Proposition 2** (Fixed-kernel correction)**.** *Let $J$ be a deterministic rollout objective with normalizable target $\pi_\lambda$ in Eq. (5). Fix $\lambda$, the surrogate (including $\mu, D, B$), the preconditioner $M \succ 0$, and the step size $\epsilon > 0$. The Markov kernel defined by the proposal in Eq. (8), surrogate drift in Eq. (13), and acceptance rule in Eq. (9) is reversible with respect to $\pi_\lambda$, regardless of the surrogate fitting error.*

*Proof.* For distinct states $z$ and $z'$, the accepted-transition density satisfies

$$\begin{aligned}\pi_\lambda(z)Q_\lambda(z' \mid z)\alpha(z, z') &= \min\{\pi_\lambda(z)Q_\lambda(z' \mid z), \pi_\lambda(z')Q_\lambda(z \mid z')\} \\ &= \pi_\lambda(z')Q_\lambda(z \mid z')\alpha(z', z).\end{aligned}$$

This is detailed balance. Rejection leaves the state unchanged, so the diagonal part of the transition also satisfies detailed balance. The full kernel is therefore reversible and leaves $\pi_\lambda$ invariant. □

# C Dexterous-Hand Evaluation Details

**Task collection.** We retarget OakInk2 [45] demonstrations to two Sharpa hands in MuJoCo Warp. The main collection contains 1352 clips, with mean and median durations of 8.16 s and 5.80 s. Seven clips were used during initial development; excluding them gives 1268 successes out of 1345 clips (94.3%). Two drawer tasks are excluded because initial collision penetrations produce nonfinite simulation states.

**Controls and objective.** The bimanual Sharpa model has $m = 56$ control channels: two 6-DoF hand bases and 44 finger joints. In Eq. (2), the object position and rotation weights are 1 and 0.3; the hand-base position and rotation weights are 0.1 and 0.03; and the finger-joint weight is 0.003. We set $w_v = 0.0001$ and $w_{\text{term}} = 1$, with no contact reward. Objects evolve under nominal gravity and contact throughout optimization. The initial hand configuration is optimized jointly with the first control window using Eq. (19).

**Optimizer settings.** Table 5 lists the optimizer configuration used for dexterous-hand evaluation.

**Table 5** GRASP-MPC settings for dexterous-hand retargeting.

| Setting | Value |
|---|---|
| Timing | 0.005 s simulation step; 0.2 s knot spacing |
| MPC window | Up to 15 knots; commit count $C = 2$ |
| Particles | $N = 2{,}000$ for full searches; ESS target $0.9N$ |
| SMC moves | 5 Markov moves per stage; at most 64 tempering stages |
| Temperature | $\beta = T_{\text{clip}}/\tau_0$; $\tau_0 = 0.006$ s |
| Prior | Appendix A |
| Surrogate | $r = 2$ temporal basis components per channel; at most 60,000 archived evaluations; cross-validated guidance shrinkage |
| Reuse and selection | Sections 5.2 and 5.3 |
| Seed and hardware | Seed 0; NVIDIA A100 80 GB GPU |

Let $T_{\text{clip}}$ denote the clip duration and $\Delta t$ the simulation timestep. We normalize each window objective by the number of steps in the full clip: $J = (\Delta t/T_{\text{clip}}) \sum_t \ell_t$, where the sum includes the stage and weighted terminal tracking terms. Setting $\beta = T_{\text{clip}}/\tau_0$ with $\tau_0 = 0.006\,\text{s}$ gives

$$\beta J = \frac{\Delta t}{\tau_0} \sum_t \ell_t \tag{25}$$

so the coefficient multiplying the window's accumulated loss does not depend on the full clip duration.

**Metrics.** For object $o$ at time $t$, let $p_{o,t} \in \mathbb{R}^3$ and $R_{o,t} \in \text{SO}(3)$ denote the simulated position and orientation, with bars marking their reference values. The tracking errors are

$$e^p_{o,t} = \|p_{o,t} - \bar{p}_{o,t}\|_2, \tag{26}$$

$$e^R_{o,t} = \|\operatorname{Log}(\bar{R}^\top_{o,t} R_{o,t})\|_2. \tag{27}$$

Here $\operatorname{Log}$ maps a relative rotation to its axis–angle vector, so $e^R_{o,t}$ is measured in radians. Following the common object-tracking metric implementation, if one reference object's mean translational displacement is below 1 mm, we use the other object's errors; otherwise, we average both objects. Averaging over the demonstrated motion segment gives the task-level errors $E_{\text{pos}}$ and $E_{\text{rot}}$. Success requires $E_{\text{pos}} < 0.1\,\text{m}$ and $E_{\text{rot}} < 0.5\,\text{rad}$; runs that terminate before completing the segment count as failures.

**HOT3D.** We use the same Sharpa models, objective, and GRASP-MPC settings for HOT3D. The evaluation contains 2,960 Aria and 2,440 Quest3 trajectories, including right-hand, left-hand, and bimanual interactions, scored at 50 Hz. As defined in Table 3, ADD-AUC integrates object-surface alignment over thresholds from 0 to 10 cm and weights trajectories equally.

**Pen-spinning PPO.** We adapt wuji-mjlab v2026.10.22 (commit `b2b98d7`) to a right Sharpa hand with 22 finger joints, keeping its original gains, armature, collision shapes, and contact pairs. The wrist is fixed and references are expressed in the palm frame, so training tracks finger and object motion without the moving-wrist dynamics of the source trajectories. Wuji-specific torque and overcurrent penalties, fingertip-force multipliers, effort-limit randomization, and pen resizing are removed; PPO, tracking rewards, adaptive sampling, the remaining domain randomization, and the disturbance curriculum are retained. The nine clips are 001–005 and 007–010 of the `01_single_axis` group. Kinematic references are the Sharpa IK trajectories, and GRASP references are the executed trajectories sampled from 200 to 50 Hz. Each condition trains one policy shared across the nine clips, with 4,096 parallel environments, 10,000 PPO iterations, and seed 0. With a single seed, the learning curves show no across-seed variability.

**Real-hand playback.** Optimized trajectories are replayed open loop at 50 Hz on OpenArm arms with Sharpa Wave hands.

# D Humanoid Evaluation Details

**Evaluation.** We evaluate robot tracking on LAFAN1, AMASS, and OMOMO, with additional object tracking on OMOMO. For clip $c$ with $n_c$ frames, let $\theta_{c,t} \in \mathbb{R}^{29}$ denote the robot joint angles, $p_{c,t}$ the pelvis position, and $r_{c,t}$ its unit quaternion; starred quantities denote the corresponding reference states. We compute

$$e_{j,c} = \frac{1}{n_c} \sum_t \|\theta_{c,t} - \theta^*_{c,t}\|_2, \qquad e_{t,c} = \frac{1}{n_c} \sum_t \|p_{c,t} - p^*_{c,t}\|_2, \tag{28}$$

$$e_{r,c} = \frac{1}{n_c} \sum_t 2 \arccos\big(\min\{1, |\langle r_{c,t}, r^*_{c,t} \rangle|\}\big) . \tag{29}$$

The joint error is the per-frame $\ell_2$ norm over all 29 joints. Joint and rotation errors are reported in radians, and translation errors in meters. The absolute quaternion inner product makes $e_{r,c}$ invariant to the signs of the two quaternion representations. Object translation and rotation errors on OMOMO use the same definitions. We compare each executed frame with its corresponding 60 Hz reference frame, average over frames within each clip, and then average the clip-level errors so that every clip has equal weight.

**Optimizer settings.** The Unitree G1 has $m = 29$ actuated joints. Table 6 lists the complete optimizer configuration used for evaluation.

**Table 6** GRASP-MPC settings for humanoid tracking. The LAFAN1/AMASS weight tuple is (pelvis translation, pelvis rotation, joint angles, contact); the OMOMO tuple is (pelvis translation, pelvis rotation, joint angles, object translation, object rotation).

| Setting | Value |
|---|---|
| Simulator | MJWarp; direct PD control; no domain randomization |
| Timing | $1/60$ s simulation step; 0.1 s knot spacing |
| MPC window | 8 knots; commit count $C = 1$; first knot pinned |
| Particles | $N = 1{,}000$; ESS target $0.9N$ |
| SMC moves | Surrogate MALA; full-prior preconditioner; 5 Markov moves per stage; no fixed stage cap |
| Surrogate | Rank 2; ridge $10^{-3}$; window parameter 3 |
| Tracking weights | LAFAN1/AMASS: $(3, 1, 1, 1)$; OMOMO: $(1, 1, 1, 1, 0.3)$; velocity: 0; intermediate terminal cost: 1 |
| Temperature convention | $\beta = 1/\tau$ |
| LAFAN1 | $\tau = 1.82 \times 10^{-4}$ ($\beta = 5.49 \times 10^3$); $(\rho, \sigma) = (0.9, 0.0142)$; initial step size 0.0026; arm scale 1 |
| AMASS | $\tau = 4.55 \times 10^{-5}$ ($\beta = 2.20 \times 10^4$); $(\rho, \sigma) = (0, 0.065)$; initial step size 0.02; arm scale 0.5 |
| OMOMO | $\tau = 1.82 \times 10^{-4}$ ($\beta = 5.49 \times 10^3$); $(\rho, \sigma) = (0, 0.065)$; initial step size 0.02; arm scale 0.5 |
| Step adaptation | Target 0.05; rate 3; minimum scale 0.1 |
| Warm start | Shift previous plan; widen the prior exploration scale toward the tail by a factor of 4 |
| Library and selection | Reuse every 2 windows with two-generation local improvement; re-evaluate the best 50 distinct candidates with 20 rollouts each and execute the lowest-mean-cost candidate |

**Evaluation clips.** For LAFAN1, every clip is the first 10.0 s and the identifier convention is `motion_subjectID`. We use `dance1` subjects $\{1, 2, 3\}$; `dance2` subjects $\{1, 2, 3, 4, 5\}$; `fallAndGetUp1` subjects $\{1, 4, 5\}$; `fallAndGetUp2` subjects $\{2, 3\}$; `fallAndGetUp3` subject $\{1\}$; `fight1` subjects $\{2, 3, 5\}$; `fightAndSports1` subjects $\{1, 4\}$; `jumps1` subjects $\{1, 2, 5\}$; `run1`

subjects $\{2, 5\}$; `run2` subjects $\{1, 4\}$; `sprint1` subjects $\{2, 4\}$; `walk1` subjects $\{1, 2, 5\}$; `walk2` subjects $\{1, 3, 4\}$; `walk3` subjects $\{1, 2, 3, 4, 5\}$; and `walk4` subject $\{1\}$, for 40 clips total. The ten AMASS clips and durations are `martial_arts_0` (20.0 s), `martial_arts_extended_0` (52.9 s), `random_stuff_0` (16.5 s), `sprint_0` (2.1 s), `sprint_1` (2.1 s), `sprint_terrain_0` (2.1 s), `cartwheels_0` (24.8 s), `fight_and_sports_0` (245.0 s), `run_to_crouch_0` (2.2 s), and `walk_to_leap_to_walk_0` (2.2 s). The two OMOMO clips are `move_largebox_0` and `move_largebox_1`, both 4.8 s.

All runs use random seed 0, eight CPU cores, and one NVIDIA A100-SXM4-80GB GPU. End-to-end wall-clock time per clip, including initialization and compilation, is 24.7 minutes on average for LAFAN1 (22.2–28.0 minutes), 89.3 minutes for AMASS (19.3–341.3 minutes), and 22.4 minutes for OMOMO (22.3–22.6 minutes).

# E Trajectory Optimization Benchmarks

These benchmarks isolate GRASP-TO's behavior under smooth dynamics and discrete contact transitions, complementing the retargeting experiments in Sections 6 and 7.

**Setup.** We compare GRASP-TO with gradient-based TSMC [42], random-walk Metropolis TSMC (RWM-TSMC), and Parallel-MPPI, a parallel baseline that runs an independent MPPI instance per particle [41]. Acrobot swing-up has smooth dynamics, allowing the gradient-based comparator to differentiate through the simulator. The planar pusher-slider selects a discrete contact edge at each step; for this task, the comparator uses a straight-through Gumbel–Softmax relaxation to obtain gradients. All reported costs are evaluated with the original simulator. Table 7 lists the problem dimensions and optimization budgets.

**Table 7** Settings for the trajectory optimization benchmarks.

| Setting | Value |
|---|---|
| Problem sizes | Both tasks: $H = 200$, $N = 100$; control dimension 1 (acrobot) or 4 (pusher-slider), giving 200 or 800 variables |
| Gradient TSMC | 3 NUTS transitions per stage; at most 8 tree doublings and 200 adaptive tempering stages |
| GRASP-TO | 99 proposals per particle and stage; at most 200 adaptive tempering stages |
| RWM-TSMC | 99 proposals per particle and stage; at most 200 adaptive tempering stages |
| Parallel-MPPI | 100 independent particles; 64 rollouts per update; 15 pusher-slider updates; acrobot updates matched to the per-particle rollout budget of rollout-based TSMC |
| Hardware | NVIDIA A100 80 GB GPU |
| Runtime boundary | From optimizer entry to return; includes each method's first-call JAX compilation; excludes common initialization, initial cost-function compilation, final re-scoring, and plotting |

**Table 8** Smooth and nonsmooth trajectory optimization benchmarks. Cost reports the mean $\pm$ standard deviation, across seeds, of the lowest-cost particle in each final population. We use seven seeds for acrobot and eleven for the pusher-slider. A pusher-slider run is solved when its lowest-cost final box pose is within 10 cm and 0.5 rad of the goal; time is measured in wall-clock seconds.

| | Acrobot | | Pusher-slider | | |
|---|---|---|---|---|---|
| Method | Cost ↓ | Time (s) | Cost ↓ | Success | Time (s) |
| Gradient-based TSMC | $2534 \pm 119$ | 2069 | $63.7 \pm 4.4$ | 11/11 | 3075 |
| **GRASP-TO** | $\mathbf{2504 \pm 98}$ | 46 | $\mathbf{54.3 \pm 2.6}$ | 11/11 | 354 |
| RWM-TSMC | $2787 \pm 210$ | 39 | $117.3 \pm 9.7$ | 1/11 | 71 |
| Parallel-MPPI | $3117 \pm 150$ | 2 | $188.6 \pm 15.2$ | 0/11 | 2 |

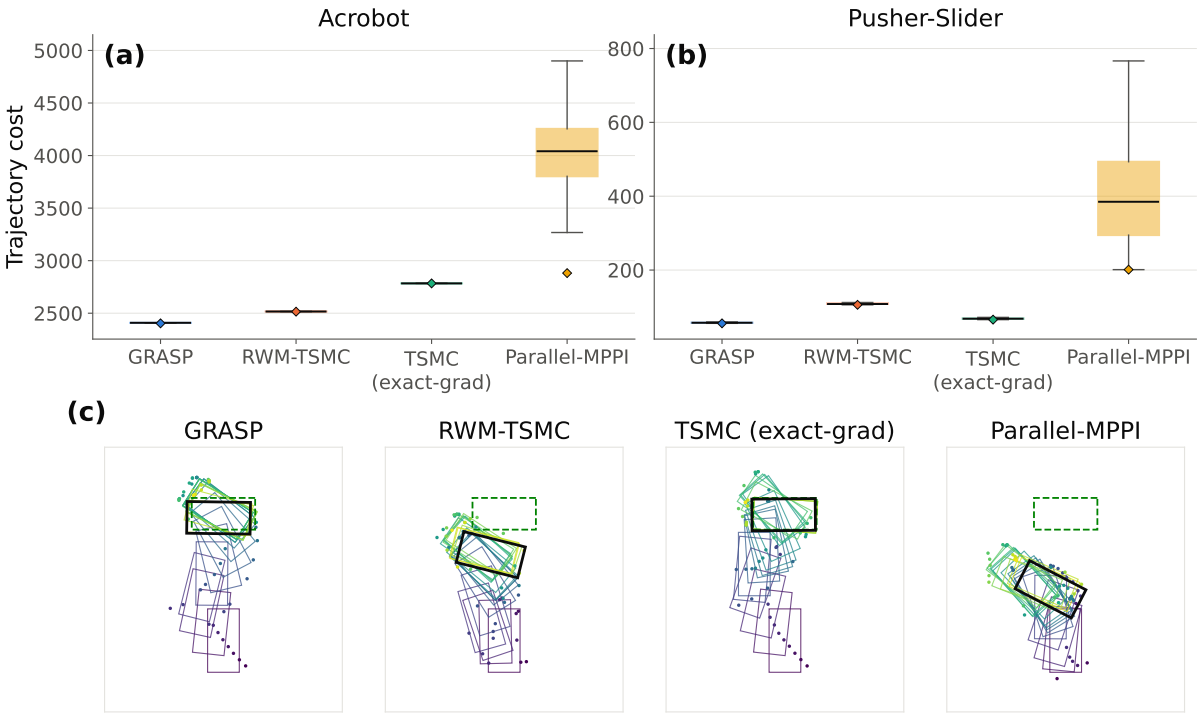


**Figure 11** Cost distributions of the final particle populations for representative runs on (a) acrobot and (b) pusher-slider; diamonds mark the best particles. (c) Pusher-slider trajectories, colored by time, with dashed goal poses. The gradient-based TSMC comparator uses simulator derivatives for acrobot and a contact relaxation for the pusher-slider.

**Results.** On acrobot, GRASP-TO reaches a cost of $2504 \pm 98$, comparable to $2534 \pm 119$ for gradient-based TSMC, with a reported runtime of 46 s versus 2069 s (Table 8). On the pusher-slider, GRASP-TO reaches $54.3 \pm 2.6$, compared with $63.7 \pm 4.4$ for the gradient-based comparator. Both solve all eleven runs. RWM-TSMC and Parallel-MPPI run faster in these configurations but attain higher costs; RWM-TSMC solves one of the pusher-slider runs and Parallel-MPPI none. Figure 11 shows representative particle populations and trajectories. Together, these results demonstrate that guidance recovered from rollout costs can support both local refinement under smooth dynamics and search across discrete contact transitions.